

COMPLETE EDITION • 2026

Debugging Field Manual

Strategies, methods, tools, and AI-powered workflows for finding and fixing anything — language-agnostic, environment-agnostic, always effective.

2026 EDITION

LANGUAGE-AGNOSTIC

29 SECTIONS

ALL LEVELS

AI-AUGMENTED

MADE IN JURGISTAN

● **BEGINNER** Core technique — universally applicable

● **INTERMEDIATE** Requires working knowledge

● **EXPERT** Deep systems-level knowledge required

Who This Guide Is For ● **BEGINNER**

Debugging is the single skill that separates engineers who ship reliably from those who don't. This guide covers the full spectrum: the cognitive frameworks for thinking about bugs, the tactical mechanics of tools, and the 2026 AI-augmented workflows that have fundamentally changed the speed of diagnosis. Whether you're stepping through Python in VS Code, tracing a distributed Rust service, or watching an LLM agent silently fail in production — this manual has you covered.



New Developer

Start here. Sections 01–08 cover the foundations before any tools matter.

→ Sections 01–08, 16, 28



Backend Engineer

Logging, tracing, database, concurrency, and production debugging.

→ Sections 07–14, 18, 23–25



Frontend Developer

Browser DevTools, network waterfall, React and performance profiling.

→ Sections 05, 10, 11, 15, 17



AI/Agent Engineer

LLM traces, agent debugging platforms, MCP debugging workflows.

→ Sections 19-22, 25



Platform / SRE

Observability stacks, distributed tracing, postmortems, incident RCA.

→ Sections 09, 18, 23-25

SECTION 01

The Debugging Mindset

Before any tool, the most important debugging instrument is the way you think.

Most debugging failures are not technical failures. They are cognitive failures: confirmation bias, tunnel vision, incorrect mental models, and emotional frustration that clouds systematic thinking. Every skilled debugger has internalized a way of reasoning before they open a terminal.

The Fundamental Principle

● BEGINNER

A bug is a discrepancy between what you believe the system does and what it actually does. Every debugging session is therefore an investigation into your assumptions. The goal is not to "find the bug" — it is to update your mental model until it matches reality. Once your model is correct, the fix is often obvious.



The Core Debugging Mantra

"The computer is always right. If the output surprises you, the surprise is about your understanding, not the machine's behavior." Adopt this as a first principle. It prevents blame-shifting and keeps you diagnostic.

Cognitive Biases That Kill Debugging Sessions

● INTERMEDIATE

BIAS	WHAT IT LOOKS LIKE	COUNTER-MOVE
Confirmation Bias	Only looking at evidence that supports your first hypothesis	Deliberately try to <i>disprove</i> your hypothesis before accepting it
Recency Bias	"It was working yesterday, so the bug must be in the code I changed today"	Use <code>git bisect</code> ; don't assume the last change is always the cause
Availability Heuristic	Jumping to a diagnosis similar to a bug you've seen before	Gather evidence before forming a hypothesis; treat each bug as unique
Sunk Cost Trap	Continuing down a wrong diagnosis path because you've already spent 2 hours on it	Set explicit time limits; be willing to restart with fresh assumptions
Tunnel Vision	Focused exclusively on one component when the real bug is in the interaction between components	Zoom out. Debug at the system boundary, not inside a single function
Moving Goalposts	Changing what "fixed" means mid-session to avoid confronting a hard problem	Write down the exact failure condition before you start. Don't move it.

The Beginner vs. Expert Debugger

● INTERMEDIATE

Beginners fix the symptom. Experts find the cause. Beginners add a guard clause; experts ask why the guard clause was needed at all. Beginners think in terms of specific lines of code; experts think in terms of state, time, and invariants. Two of the most powerful habit shifts are:

- **Think in states, not instructions.** At any given moment, what is the complete state of the system? What state was expected? What state actually exists? The bug lives in the gap.
- **Think in invariants.** What properties should always be true? A function that computes a positive number should never return negative. If it does, the invariant violation is the bug signal.



The 5-Minute Rule

Before touching any code, spend five minutes writing out what you observe, what you expect, and what you think might explain the gap. Engineers who write first consistently debug faster than those who start changing code immediately.

SECTION 02

Problem Framing & Triage

How you describe the problem determines how quickly you find it.

The most expensive mistake in debugging is starting to investigate before you have clearly defined the problem. Vague problem statements produce vague diagnoses. Precise problem statements collapse the search space dramatically.

The Problem Statement Template

● BEGINNER

Every debugging session should begin by answering these five questions in writing. "In writing" is non-negotiable — the act of typing forces precision that thinking alone does not.

```
// Problem Statement Template
```

TEMPLATE

```
// Fill this out BEFORE touching any code
```

OBSERVED: What actually happens? (Exact error message, exact output, exact behavior)

EXPECTED: What should happen? (Be specific — not "it should work")

CONTEXT: When does it happen? (Always / sometimes / only when X / since com-

mit Y)

ENVIRONMENT: Where does it happen? (OS, version, runtime, dependencies, config)

TRIED: What have you already attempted? (Don't repeat failed paths)

Bug Severity Classification

● INTERMEDIATE

SEVERITY	DEFINITION	RESPONSE TIME	WHO HANDLES
P0 — Critical	Production down, data corruption, security breach	Immediate — all hands	On-call + leads
P1 — High	Major feature broken, significant user impact	Within hours	Assigned engineer + lead review
P2 — Medium	Feature degraded, workaround available	Within days	Sprint queue
P3 — Low	Minor issue, cosmetic, edge case	Backlog	As capacity allows

Reproduction: The Most Important Step

● BEGINNER

A bug you cannot reproduce consistently is a bug you cannot reliably fix. Reproduction precedes diagnosis. Before forming any hypothesis, achieve a **Minimal Reproducible Example (MRE)**: the smallest possible piece of code or configuration that reliably demonstrates the problem.

- **Remove everything that isn't necessary.** Strip out unrelated code, configs, and data until only the essential trigger remains.
- **Document the exact reproduction steps.** Not "do the thing" — exact, numbered, ordered steps from a clean state.
- **Identify the reproduction rate.** Always? 80% of the time? Only under load? Only in production? The rate is a clue.
- **Test your MRE.** If you can't reproduce the issue with your MRE, your MRE is incomplete, not fixed.



The Heisenbug Warning

Some bugs disappear when you try to observe them — adding a print statement changes timing, attaching a debugger changes memory layout. If adding observation changes behavior, you are likely dealing with a concurrency issue, a timing-dependent bug, or an optimization difference between debug and release builds. Section 14 covers this in depth.

SECTION 03

Articulation & Rubber Ducking

The most powerful debugging tool you already own is your own voice.

Rubber duck debugging — the practice of explaining your code aloud to an inanimate object — is grounded in decades of cognitive science research on self-explanation. The act of verbalizing forces precision, exposes hidden assumptions, and activates metacognition in ways that silent reading does not.

Why It Works: The Science

● BEGINNER

When you read your own code silently, your brain fills in gaps with what you *intended* the code to do. When you explain it out loud to something that cannot infer your intent, you are forced to describe exactly what the code *actually does* — and the discrepancy between intention and description is often where the bug lives. Cognitive scientists call this the self-explanation effect: verbalizing your reasoning consistently improves accuracy and problem-solving across disciplines.

The Full Articulation Protocol

● BEGINNER

1. **State the goal.** "This function is supposed to take an array of user objects and return only those with active subscriptions."
2. **Walk through the data.** What exactly goes in? What exactly comes out? Trace a concrete example by hand.
3. **Explain each step.** Line by line. Not "this loops through the array" — "for item in users iterates over every element in the list, which at this point contains X items because..."
4. **Flag your uncertainties.** Every time you say "I think," "I assume," or "I'm not sure," that's a hypothesis to test — not a fact.
5. **State what you observe.** "But instead of returning 3 items, it returns 0. The condition must be..."



Modern Rubber Ducks

In 2026, your rubber duck doesn't have to be inanimate. Explaining your problem to an LLM — Claude, GPT-4o, Gemini — in the same methodical, step-by-step way is highly effective. The discipline of articulation produces the result, not the duck's response. But the LLM can then also ask clarifying questions.

Writing as Articulation: The Feynman Technique for Bugs

● INTERMEDIATE

Richard Feynman's learning technique translates directly to debugging: write your explanation of the bug as if explaining it to someone who knows nothing about your system. Every time you hit a point where the explanation becomes vague or hand-wavy, you've found a knowledge gap — and knowledge gaps are often where bugs hide. This is also why writing a good bug report often leads to finding the bug before submitting it.

The Complete Beginner's First Debug Session

● BEGINNER

You have an error. You have no idea where to start. Here is a complete, step-by-step protocol that works for every language and every error type.

1

Read the Error

Read the full message — especially the first and last lines. What type of error is it? What line number?

2

Find Your Code

Locate your code in the stack trace. Ignore the library frames. Focus on files you wrote.

3

Search the Exact Message

Copy the error message (without your specific values) and search it exactly. 95% of beginner errors are documented somewhere.

4

Isolate

Reduce the problem to the smallest possible reproduction. Remove everything that isn't needed to trigger the error.

5

Fix & Verify

Apply one change at a time. Verify the error is gone. Run all tests. Done.



What to Search For

When searching for an error online, remove the parts specific to your situation: file paths, variable names, line numbers. Keep the generic error type and message. `TypeError: 'NoneType'`

`object is not subscriptable` is searchable. `TypeError in /home/user/myproject/app.py`

`line 42` is not.

SECTION 04

The Hypothesis Loop

Debugging is applied science. Treat it that way.

The scientific method is the most reliable debugging methodology ever devised. Observe → Hypothesize → Predict → Test → Conclude. Debugging without hypotheses is wandering; debugging with hypotheses is navigation.

Step 1

Observe

Collect all available evidence without interpretation

Step 2

Hypothesize

Form a specific, falsifiable explanation

Step 3

Predict

State what you expect to see if the hypothesis is true

Step 4

Test

Run the smallest experiment that confirms or disproves

Step 5

Conclude

Update model; loop if hypothesis disproved

Writing Good Hypotheses

● INTERMEDIATE

A good debugging hypothesis is specific and falsifiable. "The bug is somewhere in the authentication code" is not a hypothesis — it's a region. "The JWT validation is failing because the token expiry is being compared in seconds on the server but milliseconds in the payload" is a hypothesis: it is specific, it predicts a particular failure mode, and it can be tested.



One Hypothesis at a Time

Only test one variable at a time. Changing multiple things simultaneously means you cannot determine which change produced the observed effect. This is the most common reason debugging sessions produce "fixed it, not sure why."

When to Abandon a Hypothesis

● INTERMEDIATE

Set explicit failure criteria before you test. "If I don't see evidence of X within 20 minutes, I will discard this hypothesis and form a new one." Without a time limit, confirmation bias will keep you chasing a dead end long past the point of diminishing returns. Senior engineers are distinguished less by how quickly they find the right hypothesis and more by how quickly they abandon the wrong one.

Debugging Objectively: The Evidence Log

● INTERMEDIATE

Confirmation bias is the single biggest productivity killer in debugging. You form a hypothesis, then unconsciously gather only evidence that confirms it — ignoring or dismissing contradictory data. The antidote is an explicit evidence log.

EVIDENCE ITEM	SUPPORTS HYPOTHESIS?	CONTRADICTS HYPOTHESIS?	NEUTRAL / UNKNOWN
Error appears only on POST requests	✅ Yes (if H = "write path bug")		
Error also appears in unit test with no HTTP		❌ No (not HTTP-specific)	
Bug introduced in last deploy	✅ Yes (something changed)		



Recognizing False Positives

A false positive in debugging is evidence that *appears* to confirm your hypothesis but doesn't actually. Common sources: log timestamps that don't match (log buffering delays), intermittent errors that coincidentally appeared (not caused by your change), metrics aggregations that hide the true causality. Always ask: "Is this correlation or causation?"

Which Debugging Strategy for Which Situation

● BEGINNER

SITUATION	START HERE	TOOLS
App crashes / throws exception	Read the full stack trace. Find your code frame. Inspect variable state at failure point.	Debugger, stack trace reader
Wrong output (no crash)	Add assertions at intermediate steps. Binary-search the computation. When does it first diverge from expected?	Debugger, print/log statements, unit tests
Performance degradation	Profile first. Never optimize by intuition. Identify the actual hot path before changing anything.	Profiler (py-spy, async-profiler), flame graphs
Intermittent / flaky	Maximize reproduction rate first. Log everything. Look for timing, concurrency, or state leakage.	Stress tests, load testing, <u>TSan</u> , retry-with-logging
Works locally, not in CI/production	It's an environment difference. Compare: env vars, dependency versions, OS, file paths, time zones.	Docker, act, env diff, lockfile comparison
Regression (worked before)	Use git bisect to find the commit that introduced it. Then read that commit's diff.	git bisect, git log -p

SECTION 05

Reading Errors Effectively

Error messages are documentation. Most engineers read only the last line. Don't.

The single most common debugging inefficiency is not reading the error message carefully. Developers scroll to the bottom, see the error type, and start guessing. The stack trace is a map to the bug — every frame is a clue about how execution arrived at the failure point.

Anatomy of a Stack Trace

● BEGINNER

```
# Python example — read bottom-up for cause, top-down for context
Traceback (most recent call last):
  File "app/api/users.py", line 84, in get_user_profile
    profile = fetch_from_cache(user_id)          # ← call origin
  File "app/cache/redis_client.py", line 31, in fetch_from_cache
    return json.loads(raw_value)                # ← call chain
  File "/usr/lib/python3.12/json/__init__.py", line 346, in loads
    return _default_decoder.decode(s)           # ← stdlib call
json.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
# ↑ THE ACTUAL ERROR: empty string passed to json.loads
# REAL QUESTION: why is raw_value empty? Not: why does json.loads fail on empty
string
```

PYTHON

THE READING PROTOCOL

1. **Read the error type and message first.** Understand what category of failure this is.
2. **Find your code in the stack.** The deepest frame that is *your code* (not library code) is where you have control.
3. **Read the message literally.** "Cannot read property 'id' of undefined" means something is `undefined` that you thought was an object. The property access is not the bug — the undefined is.

4. **Ask why, not how.** Not "why does `json.loads` fail on empty string" but "why is the string empty."

Error Message Taxonomy

● INTERMEDIATE

ERROR CATEGORY	WHAT IT MEANS	FIRST INVESTIGATION
NullPointerException / TypeError (undefined)	Something you assumed was non-null is null	Trace the value's origin. When and where did it become null?
IndexError / ArrayIndexOutOfBounds	Off-by-one, empty collection, or wrong assumption about length	Print the collection length immediately before the access
PermissionDenied / AccessDenied	Auth failure, file permissions, or network policy	Check credentials, roles, IAM policies, file modes
TimeoutError	Slow dependency, deadlock, or infinite loop	Which call is blocking? Add timing instrumentation
ConnectionRefused	Service not running, wrong port, firewall	<code>netstat -tlnp</code> , check service status, verify config
Segmentation Fault	Memory corruption: buffer overflow, use-after-free, dangling pointer	Run under AddressSanitizer or Valgrind immediately
ENOMEM / OOM	Heap exhausted. Often a memory leak, not a single large allocation	Profile heap over time, not at peak
Race Condition (intermittent)	Non-deterministic; time-dependent state interaction	Run under ThreadSanitizer; review all shared mutable state



Search the Exact Error Message

Paste the complete, exact error message into your search engine (or LLM). Remove only project-specific identifiers like file paths or variable names. Keep the message, type, and line numbers. You will find prior art almost every time for common errors.

SECTION 06

Binary Search Debugging

Halve the search space with every step. Find anything in $O(\log n)$ time.

Binary search is not just a data structure algorithm. It is a universal debugging strategy. Given any space of possibilities where you can test "is the bug before or after this point," binary search will find the root cause faster than any other approach.

git bisect: Finding the Breaking Commit

● INTERMEDIATE

When you know "this worked in version X and doesn't work in version Y," `git bisect` automates the binary search over your commit history. It identifies the exact commit that introduced the regression in $O(\log n)$ commit checks.

```
# Start bisecting
git bisect start
git bisect bad           # current HEAD is broken
git bisect good v2.3.1   # last known working tag
# Git checks out the midpoint commit
# Test your reproduction case, then:
git bisect good          # if this commit works
git bisect bad           # if this commit is broken
# Repeat until git reports the first bad commit
git bisect reset         # return to HEAD
```

BASH

```
# Automate with a test script (returns 0=pass, 1=fail)
git bisect run ./scripts/test_repro.sh
```

Divide and Comment: Binary Search in Code

● INTERMEDIATE

When a 500-line function produces wrong output and you don't know where, comment out the bottom half. Still broken? The bug is in the top half. Now comment out the bottom half of the top half. Repeat until isolated. This is far faster than reading the code speculatively.

Environment Binary Search

● INTERMEDIATE

Bug only happens in production? Binary search the environmental differences:

- Staging vs. production → differs in environment variables, data volume, or infrastructure?
- Replicate production env vars in staging → still different? → data or infrastructure
- Use production data snapshot in staging → narrows to data vs. infrastructure

Binary Searching Config and Environment Variables

● INTERMEDIATE

When a bug only reproduces in staging (not local) or only on one machine (not others), the cause is almost always a config or environment difference. Apply binary search to the environment itself.

1. **List all differences** between the environment where it works and where it doesn't: environment variables, OS versions, dependency versions, config files, secrets, feature flags.
2. **Bisect the list.** Copy the bottom half of differences into the failing environment. Does the bug disappear? The fix is in that half. Otherwise the other half.
3. **Common culprits in order of frequency:** incorrect `NODE_ENV` / `RAILS_ENV`, missing environment variable (evaluates to `undefined` / `None`), dependency version mismatch (`package-lock.json` ignored), different database migration state, TLS / certificate difference, timezone setting (`TZ`).



Reproduce in Docker for exact parity

Build the exact same Docker image used in CI and run it locally. `docker run --env-file .env.staging <image>` eliminates nearly all environment differences. If the bug appears in Docker locally but not in plain local dev, the environment is the bug — not your code.

Logging Strategy

Logs are the flight recorder of your application. Design them before you need them.

The most expensive debugging scenario is one where a production bug occurs and you have no observability: no logs, no traces, no metrics. Good logging is not an afterthought — it is a design discipline practiced when writing code, not after a bug report arrives.

Log Levels: What They Mean and When to Use Them

● BEGINNER

LEVEL	WHEN TO USE	WHO READS IT	IN PRODUCTION?
TRACE	Every method entry/exit, every loop iteration	Developers (local only)	Never
DEBUG	Internal state, variable values, branch decisions	Developers	Off by default; toggleable
INFO	Significant business events: request received, job started, user created	Everyone	Yes — this is your audit trail
WARN	Expected exceptional case: retry, fallback used, deprecated API called	Ops, developer on-call	Yes
ERROR	Unexpected failure that was caught. Something is broken but the app continues	On-call, alerts	Yes — should trigger monitoring
FATAL	Unrecoverable failure. Immediate shutdown required	On-call, PagerDuty	Yes — wakes someone up

Structured Logging

● INTERMEDIATE

Log as `JSON`, not free-form strings. Structured logs are queryable. Free-form strings require regex gymnastics. Every log line should answer: when, what happened, in what context, with what identifiers.

```
# X Bad: unstructured, unsearchable
logger.info("User 12345 created order 67890 for $49.99")

# ✓ Good: structured, queryable, correlatable
```

PYTHON

```
logger.info("order.created", extra={
    "user_id": "12345",
    "order_id": "67890",
    "amount_cents": 4999,
    "currency": "USD",
    "trace_id": trace_context.get_trace_id(), # Always include
    "duration_ms": elapsed
})
```

What Every Log Line Should Contain

● BEGINNER

- **Timestamp** — ISO 8601, with milliseconds, UTC timezone
- **Correlation / Trace ID** — a unique ID that follows a request through all services
- **Service / Component name** — which service produced this log
- **Log level** — structured field, not just free text
- **Event name** — a stable, queryable event identifier (e.g., `user.login.success`)
- **Relevant IDs** — `user_id`, `request_id`, `session_id`, resource identifiers



Never Log Secrets or PII

Passwords, tokens, credit card numbers, and personal identifiable information must never appear in logs. Log the presence or type of a value, not the value itself. Implement log scrubbing if using third-party logging infrastructure.

Breakpoints & Step Debugging

Time-travel through your program's execution, one step at a time.

Step debugging is the closest thing developers have to watching code execute in slow motion. Most developers use it less than they should, relying on print statements where a proper debugger would take a fraction of the time.

Core Debugger Actions

● BEGINNER

ACTION	KEY (VS CODE)	WHAT IT DOES	WHEN TO USE
Continue (F5)	F5	Run until next breakpoint or end	Moving between key points
Step Over (F10)	F10	Execute current line; don't enter function calls	Observing data flow at current level
Step Into (F11)	F11	Enter the function call on current line	Investigating function internals
Step Out (⇧ F11)	Shift + F11	Run to end of current function, re-turn to caller	Exiting a function you don't need to trace
Run to Cursor	Ctrl + F10	Run until line under cursor	Skipping to a specific point

Conditional Breakpoints

● INTERMEDIATE

A conditional breakpoint only pauses when a specified expression evaluates to true. This is indispensable for bugs that only manifest on specific inputs or after N iterations of a loop.

```
// VS Code: right-click the breakpoint dot → Edit Breakpoint → Condition
// Pause only when user_id equals a specific problematic value:
user_id === "usr_problematic_9f3k"

// Pause after 1000 iterations (Hit Count breakpoint):
// Set type to "Hit Count" with value 1000 in VS Code
```

JS

```
// Log message breakpoint (no pause – prints and continues):  
// Message: "Loop iteration {i}, value = {current_value}"
```

Watch Expressions

● BEGINNER

The Watch panel evaluates arbitrary expressions at every pause. Use it to track derived state that is not stored in a single variable, such as `users.filter(u ⇒ u.active).length` or `response.data?.items[0]?.id`.

The Call Stack Panel

● BEGINNER

When paused at a breakpoint, the call stack shows every active function frame. Click any frame to inspect its local variables. This is how you answer "how did execution get here?" without adding any instrumentation code.



REPL Evaluation at Breakpoints

The Debug Console in VS Code (and equivalent in other IDEs) is a live REPL in the paused program's scope. You can call functions, mutate variables, and test expressions against the real runtime state. Use it to test fixes before writing any code.

Distributed Tracing

When your bug crosses service boundaries, you need a map.

In a monolith, a stack trace tells you everything. In a distributed system, a stack trace tells you what failed but not why — because the real cause may live two service hops upstream. Distributed tracing gives you a single observable timeline across all services for a given request.

Core Concepts

● BEGINNER

- **Trace** — the complete record of a single request as it travels through all services
- **Span** — a single operation within a trace (an HTTP call, a DB query, a function). Spans have parent-child relationships.
- **Trace ID** — a unique identifier propagated in headers (`traceparent` per W3C spec) from service to service
- **Baggage** — additional metadata (`user_id`, feature flags) propagated alongside the trace

OpenTelemetry: The 2026 Standard

● INTERMEDIATE

OpenTelemetry ([OTel](#)) is now the industry-standard instrumentation framework. It provides vendor-neutral SDKs for traces, metrics, and logs that export to any backend (Jaeger, Zipkin, Grafana Tempo, Honeycomb, Datadog, New Relic).

```
# Python – automatic instrumentation (zero-code traces)
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap --action=install
# Then run your app:
OTEL_SERVICE_NAME=my-service \
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317 \
opentelemetry-instrument python app.py
```

PYTHON

```
# Manual span creation for custom operations
from opentelemetry import trace
```

```

tracer = trace.get_tracer("my_module")

with tracer.start_as_current_span("process_payment") as span:
    span.set_attribute("payment.amount_cents", amount)
    span.set_attribute("payment.provider", provider)
    result = charge_card(card, amount)
    span.set_attribute("payment.success", result.success)

```

Tracing Backends Comparison

● INTERMEDIATE

TOOL	TYPE	STRENGTHS	BEST FOR
Grafana Tempo + Loki + Prometheus	OSS stack	Full observability; tight Grafana integration; cost-effective	Self-hosted; cost-conscious teams
Honeycomb (https://honeycomb.io)	SaaS	Exceptional query interface; BubbleUp for anomaly detection	High-cardinality event data
Datadog APM	SaaS	All-in-one; infrastructure + APM + logs unified	Enterprise; already on Datadog
Jaeger (https://www.jaegertracing.io)	OSS	Simple, mature, widely deployed	Getting started with tracing
AWS X-Ray / CloudWatch	Cloud	Native AWS integration; no extra infrastructure	Pure AWS stacks

eBPF-Powered Zero-Code Tracing

● EXPERT

Traditional distributed tracing requires adding instrumentation libraries to every service. In 2025–2026, eBPF-based tracing eliminates this by intercepting system calls and network traffic at the kernel level. The result: distributed traces across an entire cluster with zero code changes, zero restarts, and zero SDK dependencies.

TOOL	MECHANISM	OUTPUT	BEST FOR
Grafana Beyla (https://grafana.com/docs/beyla/latest/)	eBPF uprobes on HTTP/gRPC libraries	OTel metrics + traces	Grafana stack users; first-day observability
Odigos (https://odigos.io)	eBPF + automatic language detection per pod	OTel traces to any backend	Kubernetes clusters; multi-language services
DeepFlow (https://deepflow.io)	eBPF L4–L7 packet capture + AutoTracing	Full-stack traces from network flows	Network-level debugging; DNS/TCP/TLS visibility
Kyanos (https://github.com/hengyoush/kyanos)	eBPF per-request network tracing	Per-packet latency breakdown	Debugging specific slow requests; syscall-level timings

OpenTelemetry 2026 Wire Format & SDK Advances ● EXPERT

The OTel ecosystem shipped several tracing-specific improvements in 2025–2026:

- **OTel-Arrow Phase 2** — columnar wire format based on Apache Arrow. Reduces tracing export bandwidth 5–10× compared to standard OTLP/protobuf, critical for high-throughput services generating millions of spans per minute.
- **Go Compile-Time Instrumentation** — injects tracing calls at compile time rather than runtime. Eliminates the performance overhead of reflection-based auto-instrumentation while producing identical OTel spans.
- **eBPF Go Auto-Instrumentation (Beta)** — for Go services that can't be recompiled, eBPF-based auto-instrumentation attaches to goroutine scheduling and net/http handlers to produce traces.
- **JavaScript SDK 2.0** — major rewrite with tree-shaking support, reduced bundle size (~40% smaller), and native ESM exports. Makes browser-side tracing practical without bloating client bundles.
- **Span Events → Log Records Migration** — the OTel specification is steering new instrumentation toward emitting *log records with trace context* instead of span events, enabling richer querying and correlation.



Choosing your tracing strategy in 2026

Start with eBPF zero-code (Beyla or Odigos) to get baseline traces across your cluster in minutes. Then add manual OTel SDK instrumentation only where you need richer context — custom attributes, business-logic spans, or baggage propagation. This "eBPF first, SDK second" approach minimizes engineering effort while maximizing coverage.

SECTION 10

Performance Profiling

Don't optimize what you haven't measured. Don't measure what you haven't profiled.

Performance debugging differs from correctness debugging in one critical way: intuition is almost always wrong about where the bottleneck is. Measurement must precede optimization, every time.

The Three Performance Dimensions

● BEGINNER

- **CPU** — time spent executing code. Identified by CPU profilers and flame graphs.
- **Memory** — heap allocations, GC pressure, leaks. Identified by heap profilers over time.
- **I/O** — time waiting on disk, network, or database. Identified by tracing and async profilers.



I/O Hides from CPU Profilers

If your application is slow because it's waiting on a database or external API, CPU profilers will show almost no hotspot — because the CPU is idle during the wait. Always check I/O first for latency issues. Use async/wall-clock profilers, not sampling CPU profilers.

Flame Graphs: Reading CPU Profiles

● EXPERT

A flame graph represents a call stack where width equals time spent. The x-axis is not time progression — it is alphabetical ordering of the sampled stacks. Wide bars at the top of the graph (leaf functions) are the hotspots. Work upward from wide leaf bars to find optimization opportunities.

PROFILING TOOLS BY LANGUAGE

LANGUAGE	CPU PROFILER	MEMORY PROFILER	I/O PROFILER
Python	<code>py-spy</code> (https://github.com/benfred/py-spy), <code>cProfile</code> , <code>file</code>	<code>memray</code> (https://github.com/bloomberg/memray), <code>tracemalloc</code>	<code>py-spy</code> (async), OpenTelemetry
Node.js	V8 CPU Profiler (DevTools)	Heap snapshots (DevTools)	<code>clinic.js</code>
Go	<code>pprof</code> (built-in)	<code>pprof</code> heap profile	<code>pprof</code> goroutine traces
Rust	<code>cargo-flamegraph</code> , <code>perf</code>	DHAT, Valgrind Massif	<code>tokio-console</code> (async)
Java/JVM	Async-profiler, JFR, YourKit	Eclipse MAT, JProfiler	Async-profiler (locks)
C/C++	<code>perf</code> , Valgrind Callgrind	Valgrind Massif, Heaptrack	<code>strace</code> , <code>ltrace</code>

The Profiling Decision Tree

● BEGINNER

Before profiling, you must know what kind of bottleneck you're looking for. The wrong profiler produces misleading data. CPU profiling won't show you memory leaks; memory profiling won't show you lock contention.

SYMPTOM	LIKELY BOTTLENECK	PROFILE WITH	DO NOT PROFILE WITH
High CPU, slow computation	CPU-bound: algorithm, loops, unnecessary work	CPU profiler (py-spy, async-profiler, pprof)	Memory profiler — won't show the hot function
High memory, OOM kills	Memory leak or large allocations	Heap profiler (memray, Heaptrack, Java heap dump)	CPU profiler — won't show allocation sites
High latency despite low CPU	I/O bound: database, network, disk, lock wait	Distributed tracing (Datadog APM, Jaeger); DB slow query log	CPU profiler — CPU is idle while waiting for I/O
Intermittent slowness under load	Lock contention, GC pressure, connection pool saturation	Thread profiler (async-profiler wall-clock); GC logs; TSan	Single-threaded profiler — won't capture contention
Slow only on specific inputs	Algorithm complexity ($O(n^2)$ or worse on large input)	CPU profiler targeting those specific inputs; complexity analysis	Production profiler at low sample rate — may miss the specific case

Setting Performance Baselines & Regression Thresholds

● INTERMEDIATE

"It's slow" is not actionable. "P99 latency is 320ms; our baseline is 180ms; we have a 78% regression" is. Baselines transform performance work from intuition into engineering.

1

Measure Current State

Run your workload 10+ times and record: P50, P95, P99 latency; throughput (requests/sec); memory peak; CPU utilization.

2

Record the Baseline

Store baseline values in version control alongside the benchmark suite. The benchmark is only useful if it's reproducible.

3

Set Regression Thresholds

Define what constitutes a regression: typically >10% p99 increase or >5% throughput decrease. Automate this as a CI check.

4

Run on Every PR

Run benchmarks in CI. Block merges that exceed regression thresholds. Use consistent hardware (dedicated CI runner, not shared).

5

Update After Improvements

When you intentionally improve performance, update the baseline. The baseline reflects current expectations, not historical ones.



Benchmark Noise Sources

Benchmarks on shared CI hardware are noisy. Run on dedicated hardware or accept wider thresholds (>20% regression trigger). Sources of noise: other processes competing for CPU, JIT warm-up variability, GC timing, OS scheduler decisions. Always run benchmarks at least 10 times and use the median, not the minimum.

Continuous Profiling with eBPF

• EXPERT

Traditional profilers attach to a single process for a fixed duration. Continuous profilers run permanently, sampling all processes with negligible overhead (typically <1% CPU), and store weeks of flame graph data. In 2025–2026, eBPF-based continuous profiling reached production maturity:

CNCF

PARCA

Parca (<https://www.parca.dev>) — CNCF Sandbox continuous profiler. eBPF agent samples all processes at 19Hz with ~0.5% CPU overhead. Supports flame graph diff (compare two time ranges), label-based querying, and deep integration with Prometheus for correlated metric-to-profile debugging.

GRAFANA

PYROSCOPE

Pyroscope (<https://pyroscope.io>) — Grafana's continuous profiling backend. eBPF agent + language-specific SDKs (Go, Java, Python, .NET, Ruby, Rust). Flame graphs accessible directly in Grafana dashboards alongside metrics and traces. Tag-based profiling enables per-endpoint or per-tenant views.

NETFLIX

BPFTOP

bpftop (<https://github.com/Netflix/bpftop>) — Netflix's `top`-like utility for eBPF programs. Shows per-BPF-program CPU usage, average runtime, and events/second. Essential for debugging performance of your eBPF programs themselves (including profiler agents).

OTEL

OTEL EBPF PROFILER

OTel eBPF Profiler (<https://opentelemetry.io/docs/specs/otel/profiles/>) — the OpenTelemetry Profiling SIG's reference eBPF agent. Produces profiles in the new OTel Profiles format, enabling correlation with traces and metrics in any OTel-compatible backend.



eBPF kernel compatibility

eBPF-based profilers require Linux kernel 4.18+ (5.8+ recommended for `BTF` support and CO-RE portability). On older kernels or non-Linux systems (macOS, Windows), fall back to language-specific profilers: `py-spy` (Python), `async-profiler` (Java/Kotlin), `pprof` (Go), or `perf` (Linux userspace).

System-Wide eBPF Debugging Tools ● EXPERT

Beyond profiling, eBPF powers a growing ecosystem of system-level debugging tools:

- **bpfttrace** (<https://github.com/bpfttrace/bpfttrace>) — high-level tracing language for eBPF. Write one-liner probes like `bpfttrace -e 'tracepoint:syscalls:sys_enter_read { @[comm] = count(); }'` to count reads per process.
- **Inspektor Gadget** (<https://github.com/inspektor-gadget/inspektor-gadget>) — collection of eBPF gadgets for Kubernetes debugging: trace DNS queries, snoop file access, profile syscalls, and detect anomalies — all scoped to specific pods.
- **pwru** (<https://github.com/cilium/pwru>) — packet-level tracing through the Linux kernel networking stack. Shows exactly which kernel functions touch each packet — invaluable for debugging iptables, conntrack, or CNI issues.

- **Grafana Beyla** (<https://github.com/grafana/beyla>) — also functions as a profiling data source when combined with Pyroscope, providing correlated request traces and CPU profiles in a single view.
-

SECTION 11

Network & API Debugging

HTTP is a conversation. Debug it like one — one message at a time.

Network debugging requires intercepting and examining the exact bytes being exchanged. Most application-level bugs attributed to "API issues" turn out to be malformed requests, wrong headers, or misunderstood status codes.

The Network Debugging Toolkit

● BEGINNER

curl

CLI • UNIVERSAL

The standard. Use `-v` for verbose headers, `--trace-ascii -` for full byte dump. Pipe to `jq` for JSON pretty-printing.

Postman / Bruno

GUI • API CLIENT

Full API exploration with saved collections. Bruno (OSS) stores collections as files in your repo. Postman for team collaboration.

mitmproxy (<https://mitmproxy.org>)

PROXY • INTERCEPT

Intercept, inspect, modify, and replay HTTPS traffic. Scriptable in Python. Essential for mobile/IoT API debugging.

Wireshark (<https://www.wireshark.org>)

CLI/GUI • PACKET LEVEL

Full packet capture. Use when debugging TLS handshake failures, custom protocols, or network-level latency.

httpie (<https://httpie.io>)

CLI • FRIENDLY

Human-friendly HTTP client. `http GET api.example.com/users Authorization:"Bearer token"` — readable syntax, colored output.

Browser DevTools Network

BROWSER • BUILT-IN

Waterfall view, timing breakdown, request/response headers, cookie inspection, HAR export. The first tool for any web API issue.

HTTP Status Code Diagnostic Table

● BEGINNER

CODE	MEANING	COMMON CAUSE	DEBUG PATH
400	Bad Request	Malformed JSON, missing field, validation failure	Check request body against schema; read response body for field errors
401	Unauthorized	Missing/expired/wrong auth token	Check Authorization header format; token expiry; refresh flow
403	Forbidden	Valid token, insufficient permissions	Check role/scope/IAM policy; is the resource owned by a different user?
404	Not Found	Wrong URL, resource deleted, routing misconfiguration	Verify exact URL with curl; check route registration
409	Conflict	Optimistic locking failure, duplicate resource	Check version/etag mismatch; check for duplicate create
422	Unprocessable Entity	Semantically invalid body (valid JSON, invalid content)	Read full error response body; check business rule violations
429	Rate Limited	Too many requests	Check Retry-After header; implement exponential backoff
500	Internal Server Error	Unhandled server exception	Check server logs with matching timestamp and trace ID
502/503	Bad Gateway / Service Unavailable	Upstream service down, load balancer issue	Check upstream service health; inspect proxy/LB logs
504	Gateway Timeout	Upstream too slow	Check upstream latency; database slow queries; N+1 queries

The Complete Request Trace Workflow

● INTERMEDIATE

When an API call fails or behaves unexpectedly, trace the request methodically from the client to the server and back. Each tool in this escalation ladder adds more visibility at the cost of more complexity — start simple, escalate only when needed.

1

Browser DevTools

Network tab: check status code, request headers, response body, timing. Covers 80% of web API issues.

2

curl -v

Replicate the request outside the browser. Eliminates client-side code variability. Add `-H` flags to replicate exact headers.

3

mitmproxy

Intercept and inspect HTTPS traffic from mobile apps, CLIs, or any client that can't be instrumented directly.

4

Server-Side Logs

Check what the server actually received — not what you think you sent. Use correlation IDs to match client request to server log entry.

5

Wireshark

Last resort: packet-level capture for TLS handshake failures, TCP RST analysis, or protocol-level debugging.

```
# curl -v: verbose request showing headers and response
```

BASH

```
curl -v \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{"user_id": "12345"}' \  
  https://api.example.com/v2/orders
```

```
# --trace-ascii -: dump every byte sent and received
```

```
curl --trace-ascii - https://api.example.com/health
```

```
# --resolve: test against specific IP without DNS (bypass CDN)
```

```
curl --resolve api.example.com:443:192.168.1.10 https://api.example.com/health
```

Common Network False Positives & Traps

● INTERMEDIATE

Network debugging has a particularly high false positive rate because the same symptom can have many different causes, and the most obvious explanation is often wrong.

SYMPTOM	OBVIOUS DIAGNOSIS (OFTEN WRONG)	REAL CAUSE (CHECK THIS FIRST)
Request succeeds in browser, fails in code	"API is broken"	Missing headers (User-Agent, Accept, Content-Type); cookie authentication not replicated
<u>CORS</u> error in browser	"CORS is misconfigured"	The server returned a non-2xx error before the CORS headers. Fix the underlying error first — CORS is a symptom.
DNS lookup fails intermittently	"DNS is broken"	DNS caching: stale TTL in local resolver or app-level cache. Verify with <code>dig +norecurse</code> .
<u>SSL</u> certificate error	"Certificate expired"	SNI mismatch (wrong hostname), intermediate certificate not served, or system clock skew causing "not yet valid"
Request times out only sometimes	"Network is unstable"	Load balancer routing to a bad backend; one instance unhealthy; connection pool exhaustion on specific pod
API returns 200 but wrong data	"Server-side bug"	<u>CDN</u> or proxy serving stale cached response. Bypass cache with <code>Cache-Control: no-cache</code> header.

SECTION 12

Database & Query Debugging

Most application slowdowns live in the database. Most go undetected.

Database bugs come in two flavors: correctness issues (wrong results) and performance issues (correct but slow). Both require different techniques but share the same starting point: examine the actual query being sent to the database.

EXPLAIN / EXPLAIN ANALYZE ● INTERMEDIATE

Every SQL database provides an `EXPLAIN` command that shows the query execution plan — how the database intends to execute your query. `EXPLAIN ANALYZE` actually runs the query and reports actual vs. estimated costs.

```
-- PostgreSQL – find slow queries and their execution plans
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT u.id, u.email, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at > '2025-01-01'
GROUP BY u.id;
-- Watch for: Seq Scan on large tables (missing index)
--             large actual vs. estimated row counts (stale statistics)
--             Hash Join on huge sets (memory pressure)
```

SQL

The N+1 Query Problem ● INTERMEDIATE

The most common ORM-related performance bug: fetching a list of N records, then issuing one additional query per record to fetch related data. The result is N+1 queries where 1 would suffice.

```
# X N+1: 1 query for users + N queries for orders (one per user)
users = User.all()
for user in users:
    print(user.orders.count()) # ← new DB query per user
```

PYTHON

```
# ✓ Fix: eager load with JOIN – 1 query total
users = User.all().prefetch_related('orders') # Django
# SQLAlchemy: .options(selectinload(User.orders))
# ActiveRecord: .includes(:orders)
```



Enable Query Logging in Development

Configure your ORM to log every query to the console in development. You will immediately see N+1 patterns, missing indexes, and unexpectedly expensive queries. In Django: `settings.py` `LOGGING` with `django.db.backends`. In Rails: present by default. In SQLAlchemy: `echo=True`.

Slow Query Log: Finding the Worst Offenders ● INTERMEDIATE SQL

Before optimizing, measure. Slow query logs and `pg_stat_statements` reveal real bottlenecks — not the ones you assume. Always optimize the query with the highest *total time*, not the single slowest individual run.

```
-- PostgreSQL: enable and query pg_stat_statements
-- Add to postgresql.conf: shared_preload_libraries = 'pg_stat_statements'
-- Then: CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT
  query,
  calls,
  ROUND((total_exec_time / calls)::numeric, 2) AS avg_ms,
  ROUND(total_exec_time::numeric, 2)          AS total_ms,
  ROUND((rows / calls)::numeric, 1)           AS avg_rows
FROM pg_stat_statements
WHERE calls > 10
ORDER BY total_ms DESC
LIMIT 20;

-- Reset stats after optimisation to baseline new measurements:
-- SELECT pg_stat_statements_reset();

-- MySQL / MariaDB equivalent:
```

```
-- SET GLOBAL slow_query_log = 'ON';
-- SET GLOBAL long_query_time = 0.5; -- log queries over 500 ms
-- SHOW GLOBAL STATUS LIKE 'Slow_queries';
```

Index Analysis Workflow: Before and After

● EXPERT

SQL

Adding an index without measuring is guessing. Capture the query plan before and after to verify the planner switched from a sequential scan to an index scan — and that wall time actually improved.

```
-- Step 1: Capture the baseline plan
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE u.created_at > NOW() - INTERVAL '30 days'
GROUP BY u.id, u.name;
-- Look for: "Seq Scan" on large tables – this is the target
-- Look for: "actual rows" vs "planned rows" mismatch → run ANALYZE first

-- Step 2: Add a partial index non-blocking (safe in production)
CREATE INDEX CONCURRENTLY idx_users_created_at
ON users (created_at)
WHERE created_at > '2024-01-01'; -- partial: smaller and faster

-- Step 3: Refresh the planner's statistics
ANALYZE users;

-- Step 4: Re-run EXPLAIN ANALYZE – verify "Index Scan" replaced "Seq Scan"

-- Step 5: Confirm the index is actually used in production
SELECT indexrelname, idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE relname = 'users'
ORDER BY idx_scan DESC;
-- If idx_scan stays 0 after a week, the index is unused – DROP it
```

SQL



Index creation without CONCURRENTLY blocks writes

A plain `CREATE INDEX` takes a `SHARE` lock, blocking all writes (reads still proceed) for the duration of the build. Always use `CREATE INDEX CONCURRENTLY` on production tables. The build takes longer but never blocks. See also: [PostgreSQL CONCURRENTLY docs \(https://www.postgresql.org/docs/current/sql-createindex.html#SQL-CREATEINDEX-CONCURRENTLY\)](https://www.postgresql.org/docs/current/sql-createindex.html#SQL-CREATEINDEX-CONCURRENTLY).

SECTION 13

Memory Debugging

Memory bugs are the most dangerous and the most silent.

Memory bugs fall into two broad categories: managed language memory leaks (objects retained longer than intended, preventing GC) and native memory corruption (use-after-free, buffer overflows, dangling pointers). Both require distinct tooling.

Memory Leaks in Managed Languages

● INTERMEDIATE

In garbage-collected languages, a "memory leak" occurs when objects are retained by references that are never cleared — not because memory is not freed. Common culprits: global caches that grow unbounded, event listeners never removed, closures capturing large objects, circular references in older runtimes.

```
# Python - tracemalloc: find where memory is being allocated
import tracemalloc
tracemalloc.start()

# ... run the suspect code ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')
for stat in top_stats[:10]:
    print(stat) # shows file:line and bytes allocated

# memray: production-grade heap profiler (generates flamegraphs)
# python -m memray run --output mem.bin app.py
# python -m memray flamegraph mem.bin
```

PYTHON

Native Memory Errors: AddressSanitizer (ASan)

● EXPERT

For C, C++, and Rust unsafe code, AddressSanitizer is the first tool to reach for. It instruments memory accesses at compile time and detects use-after-free, heap/stack buffer overflows, and use of uninitialized memory at runtime with precise source locations.

```
# C/C++ – compile with ASan, run normally
gcc -fsanitize=address -g -O1 -fno-omit-frame-pointer myapp.c -o myapp
./myapp # ASan will print detailed error report if memory bug triggered

# Rust – built-in sanitizer support
RUSTFLAGS="-Z sanitizer=address" cargo +nightly test -Zbuild-std --target
x86_64-unknown-linux-gnu

# ThreadSanitizer for data races (different from ASan – don't combine)
gcc -fsanitize=thread -g myapp.c -o myapp
```



Valgrind Massif for Heap Profiling

While ASan detects errors, Valgrind Massif profiles heap usage over time, producing detailed snapshots at peak usage. Use it when you need to understand *how much* memory is being used and by which code paths, not just when errors occur.

GDB: Interactive Debugging for Native Code

● EXPERT

The GNU Debugger (gdb) remains the most powerful interactive debugger for C, C++, and any language that compiles to native code. Unlike sanitizers, gdb lets you pause execution, inspect memory, evaluate expressions, and trace back from any crash — all without recompilation.

```
# Compile with debug symbols (never strip in dev)
gcc -g -O0 myapp.c -o myapp
gdb ./myapp

# Core GDB commands
(gdb) run # Start execution
(gdb) bt # Backtrace: full call stack at crash point
(gdb) frame 3 # Switch to frame 3 in the call stack
(gdb) info locals # Show all local variables in current frame
(gdb) print *ptr # Dereference and print a pointer's value
(gdb) watch sensitive_var # Break whenever this variable is written
(gdb) break myfile.c:42 # Set a breakpoint at file:line
(gdb) condition 1 x > 100 # Make breakpoint 1 conditional
(gdb) next # Step over (one source line)
(gdb) step # Step into (follow function call)
(gdb) continue # Resume until next breakpoint or crash
```

```
# Post-mortem: analyze a core dump without re-running
gdb ./myapp core.1234
(gdb) bt # Exact crash location preserved

# Attach to a running process (useful for hangs)
gdb -p 98765
(gdb) thread apply all bt # Dump stack for every thread – diagnose
deadlocks
```



gdb + ASan: the best of both worlds

Compile with both `-g -fsanitize=address`, then run inside gdb. ASan will detect the exact bug and gdb will pause there — giving you an interactive session at the precise moment of memory corruption. Set `ASAN_OPTIONS=abort_on_error=1` to ensure gdb can catch the abort signal.

SECTION 14

Concurrency & Race Conditions

Bugs that only exist when you're not looking — and disappear when you debug them.

Concurrency bugs are the hardest class of bugs to find and reproduce because they depend on the precise interleaving of operations across threads or processes — an interleaving that changes when you add any form of observation. They require static reasoning about shared state more than dynamic debugging.

Types of Concurrency Bugs

● INTERMEDIATE

BUG TYPE	DEFINITION	DETECTION TOOL
Data Race	Two threads access the same memory, at least one write, with no synchronization	ThreadSanitizer (TSan), Helgrind
Deadlock	Two or more threads each hold a resource the other needs, waiting forever	Deadlock detectors, wait-for graph analysis, timeout + stack dump
Livelock	Threads keep changing state in response to each other but make no progress	Observation of CPU at 100% with no work done; trace logs
Starvation	A thread is perpetually denied access to a shared resource by higher-priority threads	Priority analysis; queue depth monitoring
ABA Problem	Value changes from A to B back to A; CAS operation succeeds incorrectly	Code review; versioned references (stamped references)

ThreadSanitizer (TSan)

● EXPERT

```
# Compile with TSan (mutually exclusive with ASan)
gcc -fsanitize=thread -g -O1 myapp.c -o myapp
./myapp # TSan reports: "DATA RACE: write in thread T1, read in thread T2"

# Go — race detector built into the toolchain
go run -race main.go
go test -race ./...
```

SHELL



Never Debug Concurrency With Print Statements

Adding logging to concurrent code changes the timing of operations — it may cause the bug to disappear (a Heisenbug). Use TSan, proper synchronization primitives, and static analysis instead. If you must add logging, add it everywhere, not just around the suspect code.

Async/Await Pitfalls (JavaScript & Python)

● INTERMEDIATE

JS

Python

The most common async bugs are not race conditions — they are *forgotten awaits*, *sequential awaits where parallel is possible*, and *blocking the event loop with CPU work*.

```
// X Bug 1: Forgotten await – returns a Promise, not the value
async function getUser(id) {
  const user = fetchUser(id); // Promise object, not the user!
  console.log(user.name);      // undefined – user is a Promise
}

// ✓ Fix:
async function getUser(id) {
  const user = await fetchUser(id);
  console.log(user.name);      // Alice
}

// X Bug 2: Sequential awaits when parallel is possible (3× slower)
const profile = await fetchProfile(id); // waits 120 ms
const orders  = await fetchOrders(id);  // then waits 95 ms
const prefs   = await fetchPrefs(id);    // then waits 80 ms = 295 ms total

// ✓ Fix: run independent promises in parallel
const [profile, orders, prefs] =
  await Promise.all([fetchProfile(id), fetchOrders(id), fetchPrefs(id)]);
// Total: ~120 ms (the slowest one)

// X Bug 3: Blocking the event loop (all incoming requests freeze)
app.get('/process', async (req, res) => {
  const result = heavyCPUWork(req.body); // blocks Node's single thread!
  res.json(result);
});
```

JS

```
});
// ✓ Fix: offload to a Worker Thread or a job queue (BullMQ, pg-boss)
// import { Worker } from 'worker_threads';
```

Deadlock Detection and Resolution

● EXPERT

SQL

A deadlock occurs when two transactions each hold a lock the other needs, causing them to wait forever. PostgreSQL detects and automatically rolls back one party — but understanding *which queries caused it* requires looking at the lock graph.

```
-- PostgreSQL: find blocked queries and their blockers
SELECT
  blocked.pid          AS blocked_pid,
  blocked.query        AS blocked_query,
  blocker.pid          AS blocker_pid,
  blocker.query        AS blocking_query,
  blocked.wait_event_type,
  blocked.wait_event
FROM pg_locks          AS blocked_locks
JOIN pg_stat_activity AS blocked ON blocked.pid = blocked_locks.pid
JOIN pg_locks          AS blocker_locks
  ON blocker_locks.granted
  AND blocker_locks.locktype = blocked_locks.locktype
  AND blocker_locks.relation = blocked_locks.relation
JOIN pg_stat_activity AS blocker ON blocker.pid = blocker_locks.pid
WHERE NOT blocked_locks.granted
ORDER BY blocked.pid;

-- Soft cancel (allows in-flight transaction to rollback gracefully):
SELECT pg_cancel_backend(<blocker_pid>);

-- Hard terminate (use only if cancel is ignored):
-- SELECT pg_terminate_backend(<blocker_pid>);
```

SQL



Java thread dump for JVM deadlocks

Send `SIGQUIT` to the JVM: `kill -3 <pid>` (Linux/macOS) or use `jstack <pid>`. Search the output for `BLOCKED` thread states and `"waiting to lock"`. The JVM runtime also prints

"Found N deadlock(s)"

at the bottom of the dump if circular waits are detected. In Go, a full deadlock prints `all goroutines are asleep - deadlock!` and terminates the program — enabling `runtime.SetBlockProfileRate(1)` captures contention before it becomes fatal.

SECTION 15

Browser & Frontend Debugging

The browser is an extraordinarily well-instrumented runtime. Use everything it gives you.

Browser DevTools are among the most capable debugging environments ever created. Most frontend developers use 20% of their capabilities. Mastering the remaining 80% routinely cuts debugging time by an order of magnitude.

The DevTools Panels You're Underusing

● BEGINNER

PANEL	WHAT IT SOLVES	POWER FEATURE
Network	All HTTP traffic, timing, headers, payload	HAR export; Preserve Log; Throttle network; Block requests
Performance	CPU flame graphs, layout/paint thrashing, long tasks	Record → zoom into long tasks; check "Main" thread flame graph
Memory	JS heap snapshots, allocation timeline, re-tainer paths	Heap snapshot diff (before/after action) to find leaks
Application	localStorage, sessionStorage, IndexedDB, cookies, service workers	Inspect service worker cache; simulate offline
Sources	JavaScript debugger, sourcemaps, live editing	Conditional breakpoints; never-pause-here; local overrides
Console	JS errors, logs, REPL evaluation	Live expressions; filter by URL; <code>\$_</code> for last result; <code>copy()</code>

React-Specific Debugging

● INTERMEDIATE

- **React DevTools** — Inspect component tree, props, state, hooks; Profiler for render timing
- **Why Did You Render** — Library that logs unnecessary re-renders to console with full prop diff
- **React Query DevTools / Redux DevTools** — Inspect cache state, actions, time-travel debugging



Local Overrides for Production Debugging

Chrome DevTools Overrides (Sources → Overrides) lets you intercept production requests and serve your local file instead — debugging against live production data without deploying. One of the most underused debugging features in any browser.

Advanced Console API

● INTERMEDIATE

The browser console is far richer than `console.log`. These methods reduce the manual work of tracing call chains and visualizing structured data.

METHOD	PURPOSE	WHEN TO USE
<code>console.trace(label)</code>	Prints a full stack trace at the point of call	Finding where a function is being called from; tracing unexpected invocations
<code>console.table(data)</code>	Renders an array of objects as a sortable table	Inspecting arrays of API responses, state snapshots, config objects
<code>console.group(label)</code> / <code>console.groupEnd()</code>	Nests related log messages under a collapsible group	Grouping per-request or per-component logs; reducing console noise
<code>console.time(label)</code> / <code>console.timeEnd(label)</code>	Measures elapsed time between calls	Quick performance measurement without full profiler setup
<code>console.assert(cond, msg)</code>	Logs an error only if <code>cond</code> is falsy	Runtime invariant checks; catching unexpected state without throwing
<code>console.count(label)</code>	Counts how many times this label has been reached	Counting re-renders, event fires, or repeated code paths

```
// console.trace: discover WHO is calling a function
function processPayment(amount) {
  console.trace("processPayment called with", amount);
  // ... rest of logic
}

// console.group: organize related diagnostic output
console.group("AuthFlow: login attempt");
console.log("user:", email);
console.log("provider:", provider);
console.trace("call origin");
console.groupEnd();

// console.table: inspect array structures at a glance
```

JS

```
console.table(store.getState().cartItems);  
// Renders as: | id | name | qty | price |  
//           | 1 | ... | 2 | 9.99 |
```

Error Handling Design

Good error handling makes future debugging trivial. Bad error handling makes it impossible.

Debugging difficulty correlates directly with error handling quality. Code that swallows exceptions, returns magic sentinel values, and produces vague error messages makes bugs nearly impossible to trace. Error handling is not defensive programming — it is observability infrastructure.

The Error Hierarchy

● INTERMEDIATE

Distinguish between three fundamentally different categories of errors, which require different handling strategies:

CATEGORY	DEFINITION	EXAMPLES	STRATEGY
Expected Errors	Legitimate outcomes your code anticipates	User not found, payment declined, rate limit	Model as values, not exceptions. Return Result/Either types.
Programmer Errors	Bugs in your own code — invariant violations	Null pointer, wrong argument type, index out of bounds	Fail fast. Assert. Crash loudly with full context.
Infrastructure Errors	External systems failing	Database down, network timeout, disk full	Retry with backoff, circuit breaker, graceful degradation.

The Golden Rules of Error Handling

● INTERMEDIATE

- Never silence exceptions without logging.** `catch (e) { }` is not error handling — it is evidence destruction.
- Fail fast on programmer errors.** An assertion that crashes loudly is infinitely easier to debug than code that silently produces wrong output.
- Add context when wrapping errors.** "Database error" is useless. "Failed to load user profile for user_id=12345 during checkout: connection timeout" is actionable.
- Never throw from cleanup code.** Exceptions in `finally` blocks or destructors mask the original exception.

5. **Make errors structured.** Return machine-readable error objects with codes, messages, and context — not just strings.

```
# Python – error context wrapping pattern
def get_user_orders(user_id: str) → list[Order]:
    try:
        return db.query("SELECT * FROM orders WHERE user_id = %s", [user_id])
    except DatabaseError as e:
        raise ServiceError(
            message=f"Failed to fetch orders for user {user_id}",
            code="ORDER_FETCH_FAILED",
            cause=e # preserve the original exception
        ) from e # Python chained exceptions
```

Common Error Handling Mistakes

● BEGINNER

These patterns destroy debuggability. They are written by engineers who think they are handling errors but are actually just hiding them.

ANTI-PATTERN	WHY IT'S HARMFUL	FIX
Swallowing exceptions <code>except: pass</code>	Errors silently vanish. No log, no trace, no way to know the failure occurred.	Always log at minimum: <code>except Exception as e: log.error("context", exc=e)</code>
Catching too broadly <code>except Exception:</code>	Catches programmer errors (bugs) along with infrastructure errors. Masks root cause.	Catch specific exception types. Separate infrastructure from logic errors.
Log-and-rethrow without context	<code>log.error(e); raise e</code> logs the error but loses the call context at the point of logging.	Add context when wrapping: <code>raise ServiceError("Failed to load user", cause=e) from e</code>
Returning null / -1 / false on error	Callers must remember to check the return value. Null propagates silently through the call chain.	Use Result/Either types, raise exceptions, or return structured error objects.
Throwing from finally / destructor	The new exception replaces the original. The original error is lost.	Never raise from cleanup code. Catch and log any cleanup errors separately.

False Positives in Error Logs ● INTERMEDIATE

Not every error in your logs is a bug. Developing the judgment to distinguish real failures from expected noise is critical for effective debugging and alert tuning.

SCENARIO	LOOKS LIKE A BUG, BUT...	HOW TO VERIFY
User not found: 404	This is expected business behavior — user typed a bad URL, or account was deleted.	Check: is it user-triggered or system-triggered? Is rate stable or spiking?
Connection reset during deployment	During rolling deploys, in-flight requests get reset. This is expected, not a bug.	Correlate timestamp with deploy event. Rate drops to zero immediately after.
Retry storm	100 errors look like a major incident but are 1 root error amplified by retries.	Group by correlation ID or request ID. How many unique failures vs. total errors?
Health check noise	Load balancers issue aggressive 404 probes that generate log spam.	Filter logs by User-Agent or path. Health check errors ≠ user-facing errors.
Wrong log severity	ERROR logs that should be WARN (expected validation failures, rate limits, etc.)	Audit your most frequent ERROR-level log messages. Downgrade expected failures.



Build a Log Noise Baseline

Run `grep ERROR app.log | cut -d' ' -f5 | sort | uniq -c | sort -rn | head -20` to find your most frequent error messages. Any message that appears thousands of times daily is almost certainly expected behavior logged at the wrong severity — not a bug.

SECTION 17

Static Analysis & Linting

Find bugs before running a single line of code.

Static analysis examines your code without executing it, catching an entire class of bugs — type mismatches, null dereferences, unused variables, security vulnerabilities — that would otherwise require runtime conditions to surface.

Static Analysis Toolchain by Language

● BEGINNER

LANGUAGE	TYPE CHECKING	LINTING / STYLE	SECURITY SAST
Python	mypy, pyright, basedpyright	ruff (replaces flake8+isort+black)	bandit, Semgrep
TypeScript	tsc (strict mode)	eslint + @typescript-eslint	njsscan, Semgrep
Go	Compiler (strict by default)	golangci-lint (meta-linter)	gosec
Rust	Compiler (strict by default)	clippy (extensive)	cargo-audit
Java/Kotlin	Compiler + NullAway	Checkstyle, ktlint, detekt	SpotBugs, Snyk Code
Any	—	Semgrep (custom rules)	Snyk Code, CodeQL, SonarQube



Enable Strict Mode on Type Checkers

TypeScript's `strict: true` and mypy's `--strict` flag catch entire categories of bugs that lenient defaults miss. If your codebase doesn't have strict mode enabled, enabling it incrementally (file by file) is one of the highest-ROI debugging investments available.

Configuring mypy for Strict Type Checking

● INTERMEDIATE

Python

mypy's defaults are permissive. Strict mode activates the flags that catch production-class type bugs: implicit `Any` leakage, untyped function signatures, and missing return annotations.

```

# pyproject.toml – recommended strict configuration for new projects
[tool.mypy]
python_version = "3.12"
strict = true          # enables all strict flags at once
ignore_missing_imports = false # force explicit stubs (or per-module ignore)
show_error_codes = true    # show code like [attr-defined] for targeted ignores
pretty = true
warn_unused_configs = true
plugins = ["pydantic.mypy"] # if using Pydantic v2

# Install stubs for third-party libraries without inline types:
# pip install types-requests types-PyYAML types-boto3

# Run: mypy src/ --config-file pyproject.toml
# Suppress one line: # type: ignore[attr-defined]

```

Ruff: Zero-Config Fast Linting and Formatting

● BEGINNER

Python

Ruff replaces Black, Flake8, isort, Pylint, and pyupgrade in a single tool — running 10–100× faster than any of them individually.

```

# pyproject.toml – ruff configuration
[tool.ruff]
line-length = 88
target-version = "py312"

[tool.ruff.lint]
select = [
    "E",    # pycodestyle errors
    "F",    # pyflakes (undefined names, unused imports)
    "I",    # isort (import ordering)
    "N",    # PEP-8 naming conventions
    "UP",   # pyupgrade (modernise syntax for target Python)
    "B",    # bugbear (common bug patterns and design issues)
    "SIM",  # simplify (reduce code complexity)
    "RUF",  # ruff-native rules
]
ignore = ["E501"] # line length handled by formatter

[tool.ruff.format]
quote-style = "double"
indent-style = "space"

```

```
# Run: ruff check src/ --fix          (auto-fix safe issues)
# Run: ruff format src/              (format like Black)
# Run: ruff check src/ --select B    (targeted check)
```

Pre-commit Hooks: Block Bad Code at the Gate

● INTERMEDIATE

Pre-commit hooks run your static analysis before every commit. Bugs caught here never reach CI — and they never slow down a colleague's review.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.9.0
    hooks:
      - id: ruff
        args: [--fix, --exit-non-zero-on-fix]
      - id: ruff-format

  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.14.0
    hooks:
      - id: mypy
        additional_dependencies: [types-requests, pydantic]
        args: [--strict]

  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-json
      - id: check-merge-conflict
      - id: debug-statements  # catches leftover breakpoint() /
                             pdb.set_trace() calls
```

YAML

```
# Install once: pip install pre-commit && pre-commit install
# Run manually: pre-commit run --all-files
```

CI Integration: Fail Fast on Type Errors

● INTERMEDIATE

Gate your pull requests on static analysis. This is the highest-ROI step in any CI pipeline — it prevents entire classes of runtime bug before any code is deployed.

```
# .github/workflows/quality.yml
name: Code Quality

on: [push, pull_request]

jobs:
  lint-and-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: uv sync --dev

      - name: Ruff lint
        run: uv run ruff check src/ --output-format=github

      - name: Ruff format check
        run: uv run ruff format src/ --check

      - name: mypy type check
        run: uv run mypy src/ --config-file pyproject.toml
```

YAML



Other language equivalents

TypeScript:

`tsc --noEmit` + **ESLint 9** (<https://eslint.org>) flat config.

Go:

`golangci-lint run ./...` (<https://golangci-lint.run>) bundles 50+ linters in one binary.

Kotlin:

`./gradlew detekt` (<https://detekt.dev>) or `./gradlew ktlintCheck`.

Rust:

`cargo clippy -- -D warnings` as a CI gate.

Next-Generation SAST: AI-Augmented Analysis (2025–2026)

● EXPERT

Static analysis in 2026 is fundamentally different from even a year ago. The two dominant open/commercial platforms — Semgrep and SonarQube — have both shipped AI-powered capabilities that go far beyond pattern matching. Understanding what's new helps you choose the right scanning strategy for your CI pipeline.

SEMGREP 2026: MULTIMODAL & AI-POWERED

Semgrep (<https://semgrep.dev>) has evolved from a fast pattern-matching linter into a full security platform. Key 2025–2026 capabilities:

- **Semgrep Multimodal** (March 2026) — combines AI reasoning with deterministic static analysis. The engine uses LLMs to understand code intent while Semgrep rules enforce exact patterns, producing higher true-positive rates than either approach alone.
- **Custom Workflows** — define multi-step scanning pipelines that chain rules, triage logic, and autofix actions in a single declarative config.
- **Autofix (Public Beta)** — AI-generated fixes for detected vulnerabilities. Fixes are proposed as suggestions, not auto-merged, preserving human review. Powered by LLM analysis of the rule violation and surrounding context.
- **Cursor & IDE Plugins** — Semgrep integrates directly into Cursor and VS Code, surfacing findings inline as you type. No context-switching to a dashboard.
- **OWASP 2025 Coverage** — full rule coverage for the updated OWASP Top 10 2025 including AI/ML-specific risks (LLM01–LLM10).

- **Supply Chain Malware Blocking (GA)** — detects known malicious packages in `npm`, `PyPI`, and `RubyGems` at install time, preventing supply chain attacks before any code runs.
- **Community Edition 3× Performance** — the open-source engine received a major Rust rewrite, delivering 3× faster scanning on large monorepos.

SONARQUBE 2026: AI CODE ASSURANCE & THE AC/DC PARADIGM

SonarSource (<https://www.sonarsource.com>) introduced the **AC/DC** paradigm — *Agent-Centric Development Cycle* — a vision where AI coding agents and human developers form a continuous feedback loop with static analysis as the quality gate. Key capabilities in SonarQube Server 2026:

- **SonarQube Server 2026.2** — completely redesigned UI with project dashboards, AI-generated fix suggestions for detected issues, expanded language support (now 30+ languages), and unified SCA + secrets detection + SAST in a single platform.
- **AI Code Assurance** — a dedicated quality gate mode for AI-generated code. When code is flagged as AI-generated (via Git metadata or IDE signals), SonarQube applies stricter analysis rules, because AI-generated code has statistically different bug profiles than human-written code.
- **SonarQube MCP Server** — an official **Model Context Protocol** (<https://modelcontextprotocol.io>) server that lets AI agents (Claude, Copilot, Cursor) query code quality directly. During an agentic debug session, the AI can ask: "What are the critical issues in `auth/handler.py`?" and receive structured SonarQube results in context.
- **Architecture Management** — define architectural rules (dependency direction, layer boundaries) that are enforced as part of the quality gate, preventing architectural drift as teams scale.



Choosing Between Semgrep and SonarQube

Semgrep

excels at custom rule authoring (its YAML rules are remarkably expressive), supply chain security, and speed on large codebases. Best for teams that write their own security rules or need fast CI gates.

SonarQube

excels at comprehensive out-of-the-box coverage across 30+ languages, architectural enforcement, and managing technical debt at scale. Best for enterprise teams wanting a unified quality dashboard. Many teams run both: Semgrep for security-specific custom rules and supply chain, SonarQube for comprehensive quality and tech debt tracking.

SECTION 18

Monitoring & Alerting

Production observability: know about problems before your users do.

Monitoring is pre-emptive debugging. It converts bugs from "discovered by user complaint" to "discovered by alert at 2% error rate spike before any user noticed." The full observability stack is three pillars: metrics, logs, and traces.

The Three Pillars of Observability

● BEGINNER

PILLAR 1

Metrics

Aggregated numeric measurements over time. Counters, gauges, histograms. Tools: Prometheus, Datadog, CloudWatch

PILLAR 2

Logs

Discrete, time-stamped event records. High-cardinality context. Tools: Elasticsearch/Loki, Splunk, CloudWatch Logs

PILLAR 3

Traces

Request paths across services. Correlation and causality. Tools: Jaeger, Tempo, Honeycomb, Datadog APM

Error Tracking: Sentry

● INTERMEDIATE

Sentry is the de facto standard for application error tracking. Every unhandled exception is automatically captured with full stack trace, breadcrumbs (events leading up to the error), user context, and environment information. It groups similar errors, tracks resolution status, and integrates with GitHub and Jira.

```
# Python – Sentry setup (also: Node.js, Go, Java, Ruby, PHP, Rust...)
pip install sentry-sdk

import sentry_sdk
sentry_sdk.init(
```

PYTHON

```
dsn="https://[key]@sentry.io/[project-id]",
traces_sample_rate=0.1, # 10% of transactions traced
profiles_sample_rate=0.1, # 10% profiled
environment=os.environ["ENVIRONMENT"],
release=f"myapp@{VERSION}"
)

# Capture custom context for better debugging (Sentry SDK 2.x)
sentry_sdk.set_user({"id": user_id, "email": email})
sentry_sdk.set_tag("feature_flag", flag_value)
```

The Four Golden Signals (Google SRE)

● INTERMEDIATE

- **Latency** — How long requests take. Track P50, P95, P99 — not just average.
- **Traffic** — How many requests per second. Used for capacity planning and anomaly detection.
- **Errors** — Rate of failed requests (5xx, timeouts, explicit errors). Alert on relative rate increase, not absolute count.
- **Saturation** — How "full" is your service? CPU, memory, queue depth, connection pool utilization.

Alert Fatigue: Diagnosis & Cure

● INTERMEDIATE

Alert fatigue occurs when an on-call team receives so many alerts — many of which are false positives or non-actionable — that they begin to ignore or dismiss pages without investigating. The result is that real incidents go unnoticed. Alert fatigue is always a system problem, never a people problem.

1

Audit

For every alert: how often does it page? What % of pages required human action? Track for 2 weeks.

2

Classify

Categorize each alert: actionable + urgent / actionable + not urgent / informational / noise

3

Prune

Delete noise alerts immediately. Route informational alerts to dashboards, never to PagerDuty.

4

Tune Thresholds

Any alert firing more than once per week without incident requires threshold adjustment.

5

Enforce Ownership

Every alert must have a named owner who is responsible for keeping it accurate.

SLO-Driven Debugging: Burn Rate Alerts

● EXPERT

Service Level Objectives (SLOs) define your error budget: the acceptable amount of downtime or errors over a rolling period (e.g., 99.9% uptime = 43.8 min/month). Burn rate alerts trigger when you're consuming that budget faster than sustainable — giving you early warning before users are impacted.

BURN RATE	MEANING	WINDOW	ACTION
14×	Exhausting monthly budget in 2 hours	1h + 5m	Page on-call immediately
6×	Exhausting monthly budget in 5 hours	6h + 30m	Page on-call promptly
3×	Exhausting monthly budget in 10 days	1d + 6h	Ticket; investigate next business day
1×	Consuming budget at exactly scheduled rate	—	Monitor; no action needed

```
# Prometheus — 14x burn rate alert for 99.9% SLO
# Fires when 1-hour error rate would exhaust 30-day budget in 2 hours
alert: SLOBurnRateCritical
expr: |
  (
    sum(rate(http_requests_total{status=~"5.."}[1h])) /
    sum(rate(http_requests_total[1h]))
```

PROMQL

```

) > 14 * (1 - 0.999)
and
(
  sum(rate(http_requests_total{status=~"5.."}[5m])) /
  sum(rate(http_requests_total[5m]))
) > 14 * (1 - 0.999)
for: 2m
labels:
  severity: critical
annotations:
  summary: "Burning error budget 14x too fast – page immediately"

```

The Fourth Pillar: Continuous Profiling ● EXPERT

The traditional three pillars (metrics, logs, traces) are expanding. OpenTelemetry declared **Profiles** an alpha-stage fourth signal in early 2026, with stabilization tracking for 2026. Continuous profiling ties resource consumption (CPU, memory, allocations) back to individual traces and spans, answering not just *what* is slow but *which code path* is responsible and *how much* it costs.



OTel Profiles ≠ profiling tools

OTel Profiles is a *wire format and SDK API* for transmitting profiling data in the same pipeline as your metrics, logs, and traces. You still need a profiler (Parca, Pyroscope, async-profiler, etc.) to collect the data. The OTel Profiles signal standardizes ingestion so your observability backend can correlate all four signals.

eBPF-Based Observability: Zero-Code Instrumentation ● EXPERT

eBPF (extended Berkeley Packet Filter) allows attaching observability probes directly to the Linux kernel without modifying application code, container images, or sidecars. In 2025–2026, eBPF-based observability reached production maturity:

ZERO-CODE

GRAFANA BEYLA

Beyla (<https://grafana.com/docs/beyla/latest/>) — Grafana's eBPF auto-instrumentation agent. Discovers HTTP/gRPC services automatically and emits OpenTelemetry-compatible metrics and traces without any code changes. Supports Go, Java, .NET, Node.js, Python, Ruby, and Rust.

ZERO-CODE

ODIGOS

Odigos (<https://odigos.io>) — eBPF-based distributed tracing for Kubernetes. Automatically instruments all pods in a cluster and exports traces to any OTel-compatible backend (Jaeger, Tempo, Honeycomb). No code changes, no restart required.

FULL STACK

COROOT

Coroot (<https://coroot.com>) — full-stack observability platform using eBPF for automatic service map discovery, latency profiling, and anomaly detection. Pre-built dashboards with zero configuration.

AUTOTRACING

DEEPFLOW

DeepFlow (<https://deepflow.io>) — eBPF-powered AutoTracing that generates distributed traces from network traffic without touching application code. Captures L4–L7 flow data including DNS, TCP retransmissions, and TLS handshakes.

OpenTelemetry Ecosystem: 2026 Maturity Milestones

● INTERMEDIATE

OpenTelemetry is now the second most active CNCF project (after Kubernetes). Key 2025–2026 milestones that affect your observability stack:

- **Declarative Configuration (Stable)** — configure the OTel Collector, SDKs, and auto-instrumentation via a single YAML file instead of environment variables. This unifies configuration across dev, staging, and production.
- **Kotlin Multiplatform SDK** — official OpenTelemetry SDK for Kotlin Multiplatform, enabling consistent telemetry from Android, iOS (via KMP), and server-side Kotlin. Important for mobile teams wanting unified observability.
- **AI Agent Observability SIG** — a new Special Interest Group defining semantic conventions for tracing LLM/agent interactions, covering token counts, model versions, tool calls, and reasoning steps.
- **CI/CD Observability SIG** — semantic conventions for build, test, and deployment pipelines. Enables tracing a commit from PR merge through CI → staging → production with correlated spans.
- **OTel-Arrow Phase 2** — columnar wire format for high-volume telemetry transport. Reduces bandwidth 5–10× compared to OTLP/protobuf for large-scale deployments.
- **Span Events Deprecation** — span events are being replaced by *span links* and *log records* for richer causal modeling. Existing code using `span.add_event()` continues to work but new code should prefer structured logs with trace context.

AI-Assisted Debugging

LLMs as debugging partners — how to use them effectively, and where they fail.

AI coding assistants have fundamentally changed the pace of debugging for common, well-typed errors. Understanding where they excel and where they confidently produce wrong diagnoses is the key to using them as genuine accelerators rather than distractions.

What AI Assistants Are Good At

● BEGINNER

- **Error message decoding** — Explaining what a cryptic error message means in plain language
- **Common bug patterns** — Recognizing off-by-ones, null checks, type coercions, `async/await` misuse
- **Documentation synthesis** — "What does this function actually do?" with context
- **Generating hypotheses** — Brainstorming candidate causes for a described symptom
- **Fixing syntax errors** — Trivially fast for well-known languages
- **Writing test cases** — From a bug description, generate the test that would have caught it

What AI Assistants Are Bad At

● INTERMEDIATE

- **Intermittent, environment-specific bugs** — Without access to the actual environment, hypotheses are low-signal
- **Domain-specific business logic bugs** — Cannot know what the correct behavior should be for your domain
- **Performance profiling** — Cannot observe runtime behavior
- **Race conditions** — Tends to suggest obvious fixes that don't address root cause

- **Confidently providing outdated API information** — Always verify library versions and API signatures

The Effective AI Debugging Prompt

● INTERMEDIATE

```
// Template for maximum AI debugging effectivenessTEMPLATE  
  
// 1. Provide full context – don't summarize the error  
"Here is the complete error output:"  
[paste full stack trace – not just the last line]  
  
// 2. Provide relevant code  
"Here is the function that is failing:"  
[paste the minimal code that reproduces the issue]  
  
// 3. State what you've already tried  
"I have already tried: X, Y, Z. These did not fix the issue because..."  
  
// 4. Specify language, framework, and versions  
"Language: Python 3.12. Framework: FastAPI 0.111. Pydantic v2."  
  
// 5. Ask for hypotheses, not just a fix  
"What are 3 plausible causes of this error? What would I check to rule each one out?"
```

Claude Code as a Debugging Agent

● EXPERT

Claude Code (Anthropic's agentic CLI tool) can be used for multi-step debugging workflows where it reads files, executes tests, reads outputs, forms hypotheses, and proposes fixes — autonomously. Effective for: running test suites and interpreting failures, tracing execution paths through a codebase, and refactoring buggy code with full repo context.

```
# Invoke Claude Code in your project directoryBASH  
cd /path/to/project  
claude  
  
# Effective debug prompts:  
"The test test_user_checkout is failing with [error].  
Read the test, find the source of the failure, and fix it."
```

```
"This function returns incorrect totals for orders with discounts.  
Read the relevant code, trace the calculation, and identify the bug."
```

The AC/DC Paradigm: Agent-Centric Development Cycles

● EXPERT

SonarSource coined **AC/DC** — *Agent-Centric Development Cycle* — to describe the emerging pattern where AI coding agents and static analysis tools form a closed feed-back loop during debugging:

1. **Agent generates code** — AI writes a fix or new feature.
2. **SAST gate evaluates** — SonarQube (or equivalent) analyzes the generated code for bugs, vulnerabilities, code smells, and architectural violations.
3. **Agent refines** — findings are fed back to the agent as context, which iterates until the quality gate passes.
4. **Human reviews** — the developer reviews a *clean* diff that already passes all automated checks.

This pattern is enabled by **MCP integration**: the AI agent queries SonarQube via its MCP Server, reads findings, and autonomously fixes issues before the human even sees the code. The agent becomes both author and first reviewer.



AI Code Assurance

SonarQube 2026 introduced a dedicated quality gate for AI-generated code that applies stricter analysis rules. This recognizes that AI-generated code has statistically different bug profiles — more likely to miss edge cases, use deprecated APIs, or introduce subtle logic errors. Enable AI Code Assurance to automatically flag AI-authored PRs for enhanced scrutiny.

Multi-Agent Debugging Workflows

● EXPERT

Beyond single-agent debugging, 2026 saw the rise of **multi-agent architectures** for complex debugging scenarios. These combine specialized agents — each with different tool access and expertise — into an orchestrated pipeline:

- **Research Agent** — reads error logs, stack traces, and documentation. Formulates hypotheses about root cause.

- **Code Agent** — explores the codebase, traces data flow, and locates the exact lines responsible for the bug.
- **Fix Agent** — writes the minimal patch, runs tests, and iterates until green. Has write access that the research agent lacks.
- **Review Agent** — performs code review on the proposed fix, checking security, performance, and style. Read-only.

This separation of concerns mirrors human team workflows and produces higher-quality fixes than a single all-purpose agent. Tools like Claude (<https://claude.ai>) (via sub-agents), Cursor (<https://cursor.com>) (via agent mode), and Devin (<https://www.cognition.ai/devin>) support multi-agent orchestration patterns today.

SECTION 20

MCP Tools for Debugging

Model Context Protocol servers bring your observability stack directly into your AI assistant.

MCP (Model Context Protocol) servers allow AI assistants to call external tools — reading logs, querying databases, fetching traces, inspecting repos — in real time during a debugging conversation. This collapses the distance between "describe the problem to the AI" and "give the AI direct access to all evidence."

High-Value MCP Servers for Debugging

● INTERMEDIATE

GitHub MCP

CODE & HISTORY

Read file contents, search commits, inspect PRs, list issues — all within your AI debugging session. Ask "what changed in auth.py in the last 10 commits?"

Sentry MCP

ERROR TRACKING

Query Sentry issues directly from your AI session. "What are the top 5 errors in production today?" Returns full issue details with stack traces.

Datadog MCP

METRICS & LOGS

Query metrics, run log searches, inspect dashboards. "Show me error rate for the payment service in the last hour" — directly in context.

Supabase MCP

DATABASE

Run queries, inspect schema, read logs directly. Ask your AI "why is this query slow?" and have it run EXPLAIN ANALYZE in one step.

Linear / Jira MCP

ISSUE TRACKING

Correlate bug reports with code changes. "Find all issues related to the payment timeout bug filed this week."

Context7 MCP

LIBRARY DOCS

Fetch up-to-date library documentation directly into context. Prevents AI hallucinating API signatures for libraries that have changed post-training.

SonarQube MCP

CODE QUALITY

Query SonarQube code quality findings directly from an AI agent. Ask "What are the critical issues in `auth/handler.py`?" and get structured bug, vulnerability, and code smell data in context for autonomous fixing.

Playwright MCP

BROWSER DEBUG

Control a browser from an AI agent: navigate, click, screenshot, and inspect console errors. Enables end-to-end reproduction of UI bugs without manual steps.

The MCP Ecosystem in 2026

● INTERMEDIATE

The Model Context Protocol (<https://modelcontextprotocol.io>) (MCP) has become the standard interface between AI coding agents and external tools. Originally introduced by Anthropic in late 2024, MCP is now supported by:

- **Claude** (Desktop, Code, API) — native MCP support
- **ChatGPT** — MCP support added in 2025
- **VS Code / GitHub Copilot** — MCP servers configurable via `mcp.json`
- **Cursor** — first-class MCP integration with auto-discovery
- **Windsurf, Zed, JetBrains** — community and official MCP support

The MCP servers registry (<https://github.com/modelcontextprotocol/servers>) now lists 100+ servers covering observability, databases, project management, documentation, cloud providers, and security scanning. When choosing MCP servers for your debugging workflow, priori-

tize: (1) servers that surface data you currently alt-tab to check, (2) servers whose output is structured enough for an AI to act on, and (3) servers maintained by the tool vendor (official > community).

MCP-Powered Debugging Workflow

● EXPERT

```
// Example: Full production debug session with MCP tools
```

EXAMPLE

```
// 1. User reports: "Checkout is failing for some users"
```

```
// AI with Sentry MCP:
```

```
"Search Sentry for checkout errors in the last 24 hours"
```

```
→ Returns: PaymentProcessingError affecting 3.2% of users, 142 events
```

```
// AI with GitHub MCP:
```

```
"What changed in payment/processor.py in the last 5 commits?"
```

```
→ Returns: diff showing refactored retry logic 18 hours ago
```

```
// AI with Datadog MCP:
```

```
"Show error rate for payment service before and after 14:30 UTC yesterday"
```

```
→ Returns: error rate jumped from 0.1% to 3.2% at 14:37 UTC
```

```
// Correlation is complete: regression introduced by last deploy
```

MCP Debugging Setup: Step-by-Step

● BEGINNER

Get your first MCP server running in your AI assistant in under 10 minutes. The following workflow uses VS Code with GitHub Copilot or Claude Desktop.

1

Choose a Server

Start with GitHub MCP (for code context) or Context7 MCP (for up-to-date library docs). Both are free to start.

2

Install

For Claude Desktop: add to `claude_desktop_config.json`. For VS Code: add to `.vscode/mcp.json` or user settings.

3

Authenticate

Configure your API key or OAuth token as an environment variable or keychain secret. Never hardcode credentials.

4

Verify

Ask your AI: "What tools do you have available?" It should list your MCP server's tools. If not, check the server logs.

5

Test

Run a simple query: "Use the GitHub MCP to show me the last 5 commits on main." Verify the response is real data.

```
// .vscode/mcp.json - GitHub MCP config (VS Code Copilot)
{
  "servers": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      }
    }
  }
}
```

JSON

```
}  
}  
}
```

Which MCP for Which Problem ● INTERMEDIATE

BUG TYPE / SITUATION	BEST MCP SERVER	WHAT TO ASK
Regression: "it worked before"	GitHub MCP	"What changed in [file] in the last 10 commits?"
Production error spike	Sentry MCP + Datadog MCP	"Show top 5 errors in the last 24h" + "Error rate before/after [deploy time]"
Slow database query	Supabase MCP / PlanetScale MCP	"Run EXPLAIN ANALYZE on [query] and explain the bottleneck"
Library API changed / broken	Context7 MCP	"Show me the current API docs for [library function] in version [X]"
Bug report correlation	Linear / Jira MCP	"Find all issues tagged [feature] filed this week with error details"
Security vulnerability in dep	Snyk MCP / npm audit	"Check if any current dependencies have known CVEs"
Code quality gate failing	SonarQube MCP	"Show me all critical and blocker issues in the auth module"
UI rendering bug	Playwright MCP	"Navigate to /checkout, fill the form, screenshot the error state"

Debugging AI Agents

When your bug is inside a reasoning loop — and there's no stack trace.

AI agent failures are categorically different from traditional software failures. An agent may complete successfully — returning a 200 status code — while having done entirely the wrong thing. It may retrieve the wrong document, select the wrong tool, misinterpret context, or hallucinate a response, all while remaining technically functional. Traditional debugging tools are blind to this.

The Agent Failure Taxonomy

● EXPERT

FAILURE TYPE	SYMPTOM	DEBUG APPROACH
Tool Selection Failure	Agent calls wrong tool or wrong tool arguments	Trace logs showing tool call with inputs/outputs; prompt engineering
Retrieval Failure	RAG returns wrong documents; agent answers from wrong context	Log retrieved chunks; evaluate embedding similarity; inspect reranker
Reasoning Failure	Agent draws wrong conclusion from correct inputs	Inspect intermediate reasoning steps (CoT); test with smaller models
Context Overflow	Agent loses track of earlier information in long conversations	Monitor context window usage; implement memory management
Prompt Injection	Malicious input in retrieved data manipulates agent behavior	Input sanitization; system prompt separation; output validation
Loop / Thrash	Agent retries the same action repeatedly with no progress	Loop detection; iteration limits; step logging

Agent Observability Platforms

● EXPERT

LangSmith (<https://smith.langchain.com>)
LANGCHAIN • TRACING

Deep tracing for LangChain and LangGraph agents. Every LLM call, tool invocation, and retrieval step is captured. Integrated evaluation and dataset management.

Langfuse (<https://langfuse.com>)

OSS • SELF-HOSTABLE

Open-source tracing and prompt management. Self-hostable for data control. Supports all major frameworks via OpenTelemetry.

Arize Phoenix (<https://phoenix.arize.com>)

OSS • OTEL-NATIVE

OpenTelemetry-native tracing with embedding clustering. Visualize retrieval quality. Strong eval framework. Vendor-agnostic.

Braintrust (<https://braintrust.dev>)

PLATFORM • FULL LIFECYCLE

Unifies tracing, evaluation, and regression prevention. Every resolved bug becomes a permanent eval test case protecting future releases.

Helicone (<https://helicone.ai>)

PROXY • COST TRACKING

Proxy-based observability. No SDK changes. Strong cost and latency tracking per provider. Free tier for getting started.

Galileo

AUTOMATED EVALS

Automated failure detection with hallucination and safety checks. High-volume production safety monitoring.

Instrumenting an Agent with OpenTelemetry

● EXPERT

```
# Langfuse v4 – trace any LLM app with one decorator
from langfuse import get_client
from langfuse.decorators import observe

langfuse = get_client()

@observe()
def run_agent(user_query: str) → str:
    with langfuse.propagate_attributes(
        name="customer-support-agent",
        metadata={"user_tier": "premium"}
    ):
        # All nested LLM calls automatically traced
        result = agent.run(user_query)
```

PYTHON

```
langfuse.score_current_observation(  
    name="quality", value=0.9  
)  
return result
```

The AI Agent Debug Workflow

● EXPERT

When an AI agent produces wrong output, the debugging process is systematic but differs from traditional code debugging. Follow this structured investigation.

1

Reproduce

Find the exact input that triggers the failure. Is it consistent or intermittent? Confirm the failure is repeatable with the same input.

2

Classify

Use the Agent Failure Taxonomy (above): tool selection? retrieval? reasoning? context overflow? This determines which layer to inspect.

3

Inspect Traces

Open LangSmith / Langfuse. Find the failing trace. Expand each step: what prompt was sent? What was the raw LLM output? What tool calls were made?

4

Isolate the Step

Identify the exact step where things went wrong. Was the input to that step already incorrect? Or was the step itself the failure?

5

Fix & Eval

Fix the prompt, retrieval pipeline, or tool logic. Add the failing case to your eval dataset. Verify the fix doesn't regress other cases.



The "Works in Dev" Trap

Agent bugs are especially prone to environment-specific behavior. An agent that works perfectly in development may fail in production because: the retrieved documents differ (different vector store contents), the LLM temperature produces different sampling, or the context window has different content from prior conversation turns. Always test with production-representative data.

Evaluating Agent Quality: Building Evals ● EXPERT

Every resolved agent bug should become a permanent test case. An eval dataset is the CI/CD pipeline for AI agents — it prevents regressions as prompts, models, and tools change.

```
# Braintrust — create an eval from a resolved bug
import braintrust

# The resolved bug becomes a permanent regression test
eval_dataset = [
    {
        "input": {"query": "What's the refund policy for damaged items?"},
        "expected": {"contains": ["30 days", "full refund", "photo required"]},
        "metadata": {"bug_id": "AGENT-142", "resolved": "2026-03-01"}
    },
]

# Run your agent against the dataset
@braintrust.task
async def run_eval(input_data):
    result = await agent.run(input_data["query"])
    return result

# Gate your CI on eval score — block deploys if score drops
braintrust.Eval(
    name="customer-support-regressions",
    data=eval_dataset,
    task=run_eval,
    scores=[braintrust.Contains()],
)
```

PYTHON

Agentic Debug Workflows

Autonomous debugging loops that investigate, diagnose, and propose fixes.

2026 represents a fundamental shift in debugging workflows. Where debugging was once entirely manual, AI agents can now be tasked with "investigate this error and propose a fix" and autonomously read logs, query databases, inspect code history, run tests, and return a diagnosis — in minutes rather than hours.

The Autonomous Debug Loop

● EXPERT

1

Alert
Monitoring fires; error rate threshold exceeded

2

Gather
Agent pulls logs, traces, recent diffs via MCP tools

3

Hypothesize
LLM reasons over evidence, forms ranked hypotheses

4

Test
Agent runs test suite, checks repro against hypothesis

5

Report
Structured diagnosis + fix proposal posted to Slack/Linear

Tools Enabling Agentic Debugging

● EXPERT

TOOL	CAPABILITY	INTEGRATION
Claude Code	Full repo access, test execution, multi-file reasoning	CLI; VS Code; CI/CD hooks

TOOL	CAPABILITY	INTEGRATION
Cursor Agent Mode	IDE-native; codebase indexing; terminal access	VS Code fork
TestSprite	Generate tests from bug descriptions; run in cloud sand-boxes; self-heal	MCP; IDE; CI/CD
Aider	Git-aware coding agent; diff-based editing; multiple LLM backends	CLI; Python library
Devin (Cognition)	Full autonomous developer agent; can SSH into servers, run commands	API; Slack
Qodo AI	PR compliance analysis; test generation; context-aware code review	GitHub/GitLab PR integration



Human in the Loop

Agentic debugging is most effective when it handles hypothesis generation and evidence gathering — the slow, tedious parts — and surfaces a ranked shortlist of hypotheses with supporting evidence for a human engineer to validate and act on. Full autonomous fixing without human review is appropriate only for clearly bounded, low-risk changes like test fixes and formatting errors.

When to Use Agents vs Manual Debugging

● BEGINNER

Agentic debugging tools are powerful but not universally appropriate. Apply them where they amplify human judgment — not where they replace the understanding you need to own.

FACTOR	USE AGENT	DEBUG MANUALLY
Evidence volume	Thousands of log lines to correlate; multiple services; large diffs	Single file, small stack trace, obvious error type
Risk level	Low-risk: test failures, formatting, documentation bugs	High-risk: authentication, payment, data integrity, security
Reversibility	Changes are easily reverted; sandbox environment; no production impact	Production data, irreversible operations, live user impact
Domain knowledge required	Mechanical/structural bugs: type errors, import failures, test setup	Domain-specific business logic where AI lacks your context
Time pressure	Research phase: gathering evidence, forming hypotheses — let agent do the legwork	Active incident: you need to understand and own the fix personally

Safe Integration: Reviewing Agentic Fixes ● INTERMEDIATE

When an AI agent proposes a fix — whether from Claude Code, Cursor, or an autonomous debug loop — apply this review checklist before merging. The agent may fix the stated symptom while introducing a subtler problem.

UNDERSTANDING THE FIX

- ✓ I can explain exactly what changed and why it fixes the issue
- ✓ The fix addresses the root cause, not just the symptom that triggered the alert
- ✓ I've read the full diff, not just the changed lines in isolation

REGRESSION RISK

- ✓ Test suite passes with the fix applied (full suite, not targeted)
- ✓ No new assumptions introduced that could break adjacent functionality
- ✓ Edge cases analyzed: what happens with empty input, null values, maximum load?

SECURITY REVIEW

- ✓ Fix does not expand attack surface (no new unauthenticated endpoints, no new eval/exec, no SQL string concatenation)
- ✓ Any new dependencies introduced are vetted (version pinned, CVE-free)

DEPLOYMENT READINESS

✓ Can be rolled back with a single flag toggle or revert commit

✓ Monitoring in place to confirm fix holds in production

Production Debugging

When the bug is live and people are affected — calm, methodical, and fast.

Production debugging operates under pressure. The goal is not perfect understanding — it is restoring service first, then understanding the root cause afterward. Conflating the two under stress leads to longer outages and worse postmortems.

The Production Incident Hierarchy of Priorities

● EXPERT

1. **Mitigate.** Stop the bleeding. Roll back, disable the feature flag, reroute traffic. Do this before investigating.
2. **Communicate.** Update the status page and notify stakeholders. Early, honest, frequent communication reduces organizational damage.
3. **Investigate.** With service restored or mitigated, now do proper root cause analysis.
4. **Fix.** Implement and deploy the actual fix with proper testing.
5. **Postmortem.** Write it within 48 hours while memory is fresh.

Feature Flags: The Most Powerful Production Debug Tool

● INTERMEDIATE

Feature flags decouple deployment from release. If every feature is gated by a flag, any bug can be mitigated in seconds by disabling the flag — with zero deployment required. This turns a 45-minute rollback into a 30-second flag toggle.

Production-Safe Debugging Techniques

● EXPERT

- **Sampling-based tracing** — Enable verbose tracing for 1–5% of requests to reduce overhead
- **Dynamic log level control** — Toggle DEBUG logging for specific users or sessions without redeployment
- **Shadow testing** — Mirror production traffic to a shadow environment to reproduce issues safely

- **Canary analysis** — Compare metrics between canary (new) and baseline (old) pods to attribute issues to specific versions



Never Debug on Production Databases Directly

Ad hoc SQL on production — even a `SELECT` — can acquire locks, cause table scans, and degrade performance. Always test queries on a replica or staging database first. For writes, never run them manually under incident pressure.

The 15-Minute Production Incident Protocol

● EXPERT

The first 15 minutes of a production incident determine the outcome. Resist the instinct to investigate immediately — mitigate first, understand second. Every minute spent debugging instead of mitigating extends user impact.

0–2m

Assess Blast Radius

How many users? Which feature? What severity? Don't start fixing yet — understand scope.

2–5m

Mitigate

Roll back the last deploy, or disable the feature flag. Do not investigate first. Mitigation is always step 1.

5–8m

Communicate

Post an incident update: "We are aware of [issue]. Impact: [users/features]. ETA for resolution: [estimate or 'investigating']."

8–12m

Confirm Mitigation

Verify error rate is dropping. Check key metrics. Only start RCA after mitigation confirmed.

12–15m

Begin RCA

Now investigate: what triggered this? Review logs, traces, and recent diffs with service restored.



Do Not Skip Mitigation to Investigate

The most common incident error is an engineer spending 20 minutes debugging the root cause while users are impacted — when a 30-second feature flag toggle would have restored service. Always mitigate first. Root cause analysis can happen after the fire is out.

Reading Production Evidence Without Touching Production

● EXPERT

The safest production debugging principle: gather all evidence from read-only systems. Never write to production, even for diagnostic purposes.

EVIDENCE TYPE	SAFE SOURCE	UNSAFE (AVOID)
Database state	Read from a read replica; use your observability platform's DB query insights	Running ad hoc SQL on primary; using write credentials
Application logs	Stream from centralized logging (Datadog, CloudWatch, Loki) — read-only	SSH-ing into production instances; running tail on production
Request tracing	Sample from APM (Datadog traces, Honeycomb) with sampling configured ahead of time	Enabling full verbose tracing in production at 100% — may overload the system
User data inspection	Use internal tooling with audit logs; require 2-person authorization	Direct DB queries, bypassing access controls; using shared credentials
Config state	Query feature flag platform (LaunchDarkly, etc.); read config management API	Viewing production secrets or credentials directly; logging config to console

Debugging in CI/CD

If it fails in CI and not locally, the environment is the bug.

CI/CD debugging has one fundamental property that makes it hard: the environment you're running in is not the environment you're developing in. Nine out of ten "it works on my machine" problems are environment problems, not code problems.

CI Failure Taxonomy ● INTERMEDIATE

FAILURE TYPE	SYMPTOMS	RESOLUTION
Dependency mismatch	Works locally, fails in CI with version error	Pin exact versions in lockfile; ensure CI uses same lockfile
Environment variable missing	KeyError, undefined variable, connection refused	Audit all <code>os.environ</code> calls; document required env vars; use secrets management
Test ordering dependency	Tests pass individually, fail when run together	Run tests in random order (<code>pytest-randomly</code>); isolate test state; use fresh fixtures
Timing/flaky tests	Intermittent failures with no clear pattern	Add sleep → confirms timing; use proper async awaiting or polling instead
Platform difference	Fails on Linux CI, passes on macOS dev	Standardize on Docker; test in container locally
Resource exhaustion	OOM kill, timeout, disk full	Profile CI resource usage; add resource limits; use artifact caching

Reproduce CI Failures Locally ● INTERMEDIATE

```
# Reproduce GitHub Actions environment locally with act
brew install act
act -j build # runs the 'build' job locally in Docker
```

BASH

```
# GitHub Actions – debug mode: dumps all env vars and runner info
# Set repository secret: ACTIONS_STEP_DEBUG = true
# Or add step to workflow:
- name: Debug info
```

```
run: |
  env | sort
  docker info
  cat /etc/os-release
```

The Flaky Test Triage Workflow ● INTERMEDIATE

Flaky tests — tests that sometimes pass and sometimes fail without code changes — are one of the most destructive forces in a CI/CD pipeline. Every false failure costs engineer time and erodes trust in the test suite. Triage them systematically rather than ignoring them.

1

Classify

Is the failure: timing-dependent? Order-dependent? Resource-dependent (CPU/memory)? Network-dependent? Classification determines the fix.

2

Isolate

Run the test in isolation 50 times: `pytest tests/test_foo.py -x --count=50` (requires `pip install pytest-repeat`). Does it flake alone? If not, it's order-dependent.

3

Quarantine

If you can't fix immediately: mark as skipped with a linked issue. Do not leave it blocking CI. Track quarantined tests on a dashboard.

4

Fix Root Cause

Timing: replace `sleep()` with polling. Order: add setup/teardown isolation. Resource: mock external dependencies.

5

Verify

Run the test 100 times in CI to confirm. Re-enable and monitor for 2 sprints before declaring fixed.

FLAKY PATTERN	SYMPTOM	FIX
Timing dependency	Passes slowly, fails when CPU is loaded	Replace <code>sleep(n)</code> with <code>wait_for(condition, timeout)</code>
Shared state	Passes alone, fails after specific other tests	Add proper setup/teardown; use test fixtures; reset global state
Port collision	Fails with "address already in use" when tests run in parallel	Use port 0 (OS-assigned) or ephemeral test ports; use unique test namespaces
External dependency	Fails when network/DB is slow or unavailable	Mock the external dependency; use contract testing instead

CI Debugging Step-by-Step

● BEGINNER

Your build passed locally but failed in CI. Here is the complete beginner workflow for diagnosing CI failures — no CI expertise required.

1. **Read the full CI output.** Scroll to the first ERROR line, not the summary. The real error is always before the "Build failed" message.
2. **Identify the failure type.** Is it a compilation error? A test failure? A linting error? A timeout? Each requires a different fix strategy.
3. **Check the environment diff.** What does CI have that your machine doesn't (or vice versa)? Compare: OS, language version, environment variables, and installed dependencies.
4. **Try to reproduce locally.** Run `act` (for GitHub Actions) or equivalent. Use Docker to match the CI environment exactly if needed.
5. **Add debug output.** Enable CI debug mode (`ACTIONS_STEP_DEBUG=true` for GitHub Actions). Add `env | sort` as an early step to dump all environment variables.
6. **Fix one thing at a time.** Change one thing — pin a dependency version, add a missing env var — then re-run CI. Don't batch changes or you won't know what fixed it.

SECTION 25

Root Cause Analysis & Postmortems

The bug is fixed. Now make sure it never happens again.

A bug that is fixed without root cause analysis will recur in a different form. RCA is the practice of going beyond "what failed" to understand "why the system allowed it to fail" — and then changing the system, not just the symptom.

The Five Whys

● BEGINNER

Asking "why?" five times in succession typically moves from symptom to root cause, traversing organizational, process, and systemic layers that the initial technical cause obscures.



Five Whys Example

Problem:

Production outage — users cannot log in.

Why 1:

The authentication service returned 503 errors.

Why 2:

The JWT secret key environment variable was empty.

Why 3:

The Kubernetes secret was not mounted in the new deployment.

Why 4:

The deployment manifest did not include the secret mount (copy-paste error).

Why 5:

We have no validation that required secrets are present at startup.

Root cause fix:

Add startup validation that fails fast if required env vars are missing.

Blameless Postmortems

● INTERMEDIATE

The goal of a postmortem is systemic improvement, not accountability assignment. Google SRE, Netflix, and Etsy have all pioneered the blameless postmortem culture: the assumption is that engineers do the best job they can with the information and tools available to them. When a bug occurs, it is evidence of a system failure — inadequate testing, insufficient observability, unclear process — not evidence of individual incompetence.

POSTMORTEM TEMPLATE

```
## Postmortem: [Incident Title] - [Date]
```

MARKDOWN

Impact: [Duration] · [Users affected] · [Business metric impact]

Timeline:

HH:MM — [Event]

HH:MM — [Detection]

HH:MM — [Mitigation]

HH:MM — [Resolution]

Root Cause: [One clear sentence]

Contributing Factors:

- [What made this possible]
- [What made this hard to detect]
- [What made this hard to fix quickly]

What Went Well:

- [Honest positives – detection speed, communication, etc.]

Action Items:

- [Specific, owned, time-bound changes to prevent recurrence]
- Owner: [Name] • Due: [Date]

Building the Incident Timeline

● INTERMEDIATE

The incident timeline is the backbone of any postmortem. An accurate timeline reveals detection lag (how long the incident ran before anyone knew), response lag (how long between detection and action), and cascading effects. Reconstructing it accurately requires working from multiple sources.

1

Anchor with Objective Data

Start with machine timestamps: monitoring alerts, deployment logs, database transaction logs. These can't be misremembered.

2

Fill with Chat History

Pull Slack / Teams incident channel logs. Note when people first mentioned symptoms vs. when they escalated.

3

Layer Human Memory

Interview responders within 24 hours while memory is fresh. Flag human-recalled events as approximate.

4

Identify Key Gaps

Where are there gaps in the timeline? Missing monitoring? No alert? Manual detection? Each gap is a postmortem action item.

5

Validate Against Metrics

Cross-check the timeline against error rate charts, latency graphs, apdex scores. Timestamps should match.



Timeline Anti-Patterns

Hindsight bias:

"We noticed X at T+5" is often false — the alert fired at T+5, but no one acted until T+20. Log when people *acted*, not when they *could have* noticed.

Missing the start:

The incident often started before the first alert. Look for precursor signals in monitoring — a slow ramp-up in errors, a gradual memory increase — that appeared before the threshold was crossed.

Action Items That Get Done

● INTERMEDIATE

The most common cause of postmortem failure is vague action items that no one owns and no one completes. An action item that doesn't prevent the next incident is worse than no action item: it creates false confidence.

BAD ACTION ITEM	WHY IT FAILS	GOOD ACTION ITEM
"Improve monitoring"	Vague, unowned, unmeasurable	"Add alert: error rate > 1% for > 5min. Owner: @alice. Due: Feb 14."
"Be more careful with deploys"	Human behavior change — not systemic	"Add automated canary analysis gate to deploy pipeline. Owner: @bob. Due: Feb 21."
"Fix the code"	No specifics, no owner, no test requirement	"Add validation that JWT secret is non-empty on startup; add unit test. Owner: @carol. Due: Feb 14."
"Communicate better during incidents"	No mechanism for enforcement	"Create incident runbook with communication template; add to on-call guide. Owner: @dave. Due: Feb 28."



The Action Item Review Cycle

Schedule a 30-minute postmortem review 4 weeks after the incident to verify all action items were completed. Track completion rate across incidents as an organizational health metric. Teams with <70% completion rates have a process problem, not a people problem.

SECTION 26

The 2026 Toolkit Reference

Every tool you need, organized by what problem it solves.

This is a reference index of the full debugging ecosystem in 2026, organized by category. All entries are actively maintained and production-tested.

Universal Debugging

● BEGINNER

TOOL	PURPOSE	FREE?	URL
VS Code	IDE debugger with multi-language step debugging, conditional breakpoints, REPL	Yes	code.visualstudio.com (https://code.visualstudio.com)
Cursor	AI-native VS Code fork with agent mode for autonomous debugging	Freemium	cursor.sh (https://cursor.sh)
JetBrains IDEs	Best-in-class debugger for Java/Kotlin/Python/Go; deep framework integration	Paid	jetbrains.com (https://www.jetbrains.com)
ChatDBG	LLM integration for GDB, LLDB, and pdb — ask questions about crash state	OSS	GitHub (https://github.com/plasma-umass/ChatDBG)
git bisect	Binary search across commits to find regression origin	Yes (git)	Built into git

Observability & Monitoring

● INTERMEDIATE

TOOL	CATEGORY	TIER
Sentry	Error tracking, session replay, performance monitoring	Freemium SaaS / OSS self-host
Datadog	Full-stack APM, logs, metrics, synthetics	Paid SaaS
Grafana + Prometheus + Loki + Tempo	OSS observability stack — metrics + logs + traces	OSS self-host
Honeycomb	High-cardinality event-driven observability	Freemium SaaS
OpenTelemetry (https://opentelemetry.io)	Vendor-neutral instrumentation SDK (use with any backend)	OSS
New Relic	Full-stack observability; generous free tier	Freemium SaaS

Network & Protocol

● INTERMEDIATE

TOOL	USE CASE	PLATFORM
mitmproxy	HTTP/HTTPS interception and modification	All (Python)
Wireshark	Packet capture and deep protocol analysis	All
curl	HTTP client for manual API testing	All (CLI)
Bruno	OSS Postman alternative; stores collections as files	All
httpie	Human-friendly CLI HTTP client	All (Python)
Charles Proxy	Mobile HTTPS interception with GUI	macOS/Win

Memory & Native Debugging

● EXPERT

TOOL	WHAT IT FINDS	LANGUAGE
AddressSanitizer (ASan)	Buffer overflows, use-after-free, heap corruption	C, C++, Rust
ThreadSanitizer (TSan)	Data races, deadlocks	C, C++, Go
Valgrind (Memcheck)	Memory errors, memory leaks (slower but thorough)	C, C++
memray	Python heap profiler; flamegraphs; production-safe	Python
Heaptrack (https://github.com/KDE/heaptack)	Heap profiler with flamegraphs; faster than Valgrind	C, C++

AI Agent Debugging

● EXPERT

TOOL	SPECIALITY	OSS?
LangSmith	LangChain/LangGraph traces; deep LLM call inspection	No (SaaS)
Langfuse	Self-hostable; OTel-based; framework-agnostic	Yes
Arize Phoenix	OpenTelemetry-native; embedding visualization; strong evals	Yes
Braintrust	Tracing + evaluation + CI gating unified	No (SaaS)
Helicone	Proxy-based; zero-SDK cost and latency tracking	OSS core

eBPF & Continuous Profiling

● EXPERT

TOOL	SPECIALITY	OSS?
Parca	CNCF continuous profiler; eBPF agent; flame graph diffs; Prometheus integration	Yes
Pyroscope	Grafana continuous profiling; eBPF + SDK agents; tag-based profiling	Yes (AGPLv3)
Grafana Beyla	Zero-code auto-instrumentation; eBPF; OTel metrics + traces	Yes
bpfttrace	High-level eBPF tracing language; one-liner probes for syscalls and functions	Yes
Inspektor Gadget	Kubernetes eBPF debugging gadgets; pod-scoped tracing	Yes
bpftop	Netflix; top for eBPF programs — CPU, runtime, events/sec	Yes
Odigos	Zero-code distributed tracing for Kubernetes; auto-detect languages	Yes
DeepFlow	AutoTracing from L4–L7 network flows; DNS/TCP/TLS visibility	Yes (AGPLv3)

MCP Servers for Debugging

● INTERMEDIATE

MCP SERVER	DATA EXPOSED	OFFICIAL?
GitHub MCP	Repos, PRs, issues, commits, code search	Yes
Sentry MCP	Error events, breadcrumbs, release tracking	Yes
Datadog MCP	Metrics, traces, logs, monitors, dashboards	Yes
SonarQube MCP	Bugs, vulnerabilities, code smells, quality gates	Yes
Supabase MCP	Database queries, migrations, edge functions, logs	Yes
Playwright MCP	Browser control, screenshots, console, network	Yes
Context7 MCP	Up-to-date library documentation	Community

SECTION 27

Anti-Patterns & Traps

The things that feel like debugging but aren't.

Years of debugging reveal recurring failure patterns — habits that feel productive but actively impede resolution. Recognizing these anti-patterns in your own workflow is the first step toward replacing them with the disciplined prac-

tices covered throughout this manual.

The 10 Most Expensive Debugging Anti-Patterns

● BEGINNER

#	ANTI-PATTERN	WHY IT'S HARMFUL	ALTERNATIVE
1	Shotgun debugging	Changing multiple things at once makes it impossible to know what fixed the issue	Change one variable at a time; document each result
2	Print-statement archaeology	Adding print statements produces too much noise; requires code changes; pollutes production logs	Use a real debugger with breakpoints; use structured logging
3	Fixing the symptom	The bug reappears in a different form; you've added complexity without solving the underlying issue	Always ask "why did this happen?" at least once more
4	Cargo-culting Stack Overflow	Pasting solutions without understanding them introduces new bugs and tech debt	Understand what the fix does before applying it; test it
5	Debugging in production	High risk; affects real users; creates pressure that degrades judgment	Reproduce locally; use staging; use feature flags
6	Assuming the new code is wrong	The regression is in the new code — but the cause is often in older code that the new code exposed	Bisect to find the interaction; don't assume recency implies causality
7	Fixing without a test	The bug will recur; no regression protection; future authors will reintroduce it	Write the failing test first; then fix; confirm test passes
8	Asking for help too early	You shortcut your own learning; develop learned helplessness; waste others' time	Spend 20–30 minutes with the problem; articulate it fully first
9	Asking for help too late	Hours lost on a wrong hypothesis that a colleague would have spotted in two minutes	Set a timer; if still stuck after 30 min, seek input — don't be heroic
10	Trusting AI fixes blindly	AI-suggested fixes are plausible but often subtly wrong; they can introduce new bugs while fixing the stated issue	Understand the fix; run the full test suite; review the diff

Organizational & Process Anti-Patterns

● EXPERT

Anti-patterns are not limited to individual technique. The most expensive bugs often survive because of organizational habits that inhibit clear diagnosis.

When to escalate rather than continuing solo: If you have been on the same hypothesis for more than 30 minutes with no new evidence, you are probably stuck — not progressing. The escalation trigger is not a sign of weakness; it is systematic process. The 30-minute rule: articulate the problem and your evidence, then seek input. Rubber-duck debugging alone often reveals the answer before you need to interrupt anyone.

ANTI-PATTERN	ORGANIZATIONAL SIGNAL	SYSTEMIC FIX
Hero debugging culture	One person always holds the context; single points of failure	Pairing rotation; runbook documentation; blameless postmortems
No reproduction environment	Bugs can only be reproduced in production	Maintain a staging environment that mirrors prod; use feature flags
Alert fatigue	Team ignores pages because false positives are too common	Ruthless alert pruning; SLO-based alerting; on-call rotation reviews
Fix-without-learning loop	Same class of bug recurs across quarters with no systemic response	Require postmortems for all P1s; track bug taxonomy; act on patterns
Blame-first culture	People hide bugs or minimize scope to avoid accountability	Adopt blameless postmortems; measure MTTR, not blame attribution

The False Positive Trap

● INTERMEDIATE

False positives in debugging are situations that look exactly like bugs but are actually correct behavior, expected errors, or environment noise. Mistaking them for bugs wastes hours and builds incorrect mental models of the system.

SITUATION	LOOKS LIKE A BUG	ACTUALLY IS	HOW TO VERIFY
Eventual consistency lag	Read-after-write returns stale data	Expected in distributed systems with replica lag	Add a delay; read from primary. If read is consistent, it's lag.
Cache miss on first request	First page load is 10× slower	Cold cache — expected behavior	Warm the cache; repeat. If second request is fast, it's cache cold start.
Race condition in test	Test fails intermittently with no pattern	Test environment has shared state or timing dependency	Run test 100 times; add delays; check test isolation.
Configuration not applied yet	Changed config has no effect	Service restart, reload, or cache invalidation needed	Restart the service; check for config caching layers.
Expected validation rejection	Seeing 400 errors in logs	Client sending invalid data — correct behavior by server	Check if the 400 rate is stable. Is it correlated with user actions?

Recognizing Your Own Cognitive Biases

● INTERMEDIATE

Debugging is a cognitive task performed under uncertainty. Every engineer carries biases that systematically distort their debugging judgment. Recognizing them is the first step to counteracting them.

BIAS	HOW IT MANIFESTS IN DEBUGGING	ANTIDOTE
Confirmation Bias	Only noticing evidence that supports your first hypothesis; dismissing contradictory logs	Write an explicit evidence log. List confirming AND contradicting evidence equally.
Anchoring Bias	First diagnosis sticks even after evidence against it accumulates; hard to let go	Set a timer: if hypothesis isn't confirmed in 20 minutes, explicitly abandon it and start fresh.
Recency Bias	"The last thing changed must have caused it" — but the real cause was older code exposed by the new code	Use git bisect to find the actual regression commit, not just the most recent one.
Sunk Cost Fallacy	"I've spent 3 hours on this hypothesis, I can't abandon it now"	The hours spent are gone regardless. The only question is: what's the fastest path to the truth from here?
Availability Bias	Assuming the bug is in the area you know best or the area you debugged last	Follow the evidence, not your intuition. Trace from the error, not from component familiarity.



The Bias Check Question

Before spending another 20 minutes on a hypothesis, ask yourself: "If I had to explain this hypothesis to a colleague right now, could I cite at least two independent pieces of evidence that support it?" If the answer is no, you may be running on bias rather than evidence.

SECTION 28

Master Debugging Checklist

The complete pre- and post-debug quality protocol.

Use this protocol as both a preflight checklist and a post-fix audit. Check each item in sequence; unchecked boxes are clues, not failures. Adapt the order to your situation — a production outage skips straight to evidence collection, while a local regression works through the full protocol top-to-bottom.

Pre-Debug Protocol

● BEGINNER

BEFORE YOU START

PROBLEM DEFINITION

- ✓ Written problem statement with observed vs. expected behavior
- ✓ Exact reproduction steps from a clean state documented
- ✓ Reproduction rate identified (always / % of time / only under X condition)
- ✓ Environment identified: OS, language version, framework version, dependencies
- ✓ Earliest known-good version identified for regression investigation

MINDSET CHECKS

- ✓ Confirmed: not changing multiple things simultaneously
- ✓ Confirmed: hypothesis is specific and falsifiable (not "the issue might be in auth")
- ✓ Time limit set: if not resolved in 30 minutes on current hypothesis, try a different one

- ✓ Bias audit: am I looking for what confirms my first hypothesis, or evidence in general?

Investigation Mechanics

● INTERMEDIATE

DURING INVESTIGATION

EVIDENCE COLLECTION

- ✓ Full error message and stack trace read carefully (not just the last line)
- ✓ Relevant logs checked with timestamp correlation to reproduction
- ✓ Recent code changes checked (git log/diff since last known-good state)
- ✓ Environment variables and configuration checked
- ✓ Dependency versions verified against expected

DEBUGGING MECHANICS

- ✓ If using a debugger: checked variable state at the point of failure
- ✓ If using logs: added correlation ID to trace across components
- ✓ If API issue: used curl/httpie to isolate from client-side code
- ✓ If database issue: ran EXPLAIN ANALYZE on the slow/incorrect query
- ✓ If performance issue: profiled — not optimized by intuition
- ✓ If intermittent: checked for concurrency issues; tried TSan if applicable

AI-ASSISTED

- ✓ Provided full context (error + code + versions) — not a summary
- ✓ Stated what's already been tried so AI doesn't repeat
- ✓ Verified library version compatibility of any AI-suggested API calls

- ✓ Understood AI-suggested fix before applying it

Post-Fix Quality Gates

● INTERMEDIATE

AFTER FIXING

FIX QUALITY

- ✓ Fix addresses root cause, not just symptom
- ✓ Test written that would have caught this bug (or would catch a regression)
- ✓ Fix tested in isolation AND in full integration
- ✓ Fix does not introduce new assumptions that could become the next bug

KNOWLEDGE TRANSFER

- ✓ Bug cause documented in ticket/PR description
- ✓ If PO/PI: postmortem written within 48 hours
- ✓ If systemic: follow-up ticket created for root cause prevention
- ✓ If tooling gap: monitoring/alerting improved to detect this class of failure earlier

PREVENTION

- ✓ Static analysis catches this class of error going forward?
- ✓ Is there a code review checklist item to prevent recurrence?
- ✓ Has the team been informed if the bug represents a pattern risk?

Situation-Specific Checklists

● BEGINNER

These micro-checklists get you unstuck in the first 5 minutes for the five most common debugging scenarios. Use the master checklist above for thorough investigation; use these when you need a fast start.

APP CRASHES ON STARTUP

- ✓ Read the first error in the crash output — not the last

- ✓ Check environment variables: all required ones present and correct?
- ✓ Check dependency versions: any recent upgrade that could break compatibility?
- ✓ Check configuration files: valid syntax? Required fields present?
- ✓ Check port conflicts: is the required port already in use?

API RETURNS WRONG DATA

- ✓ Test the API endpoint directly with curl or Bruno — isolate from frontend code
- ✓ Check the request payload: is the correct data being sent?
- ✓ Check authentication: are credentials valid and not expired?
- ✓ Check the database: does the underlying data match what you expect?
- ✓ Check caching layers: is stale data being returned from cache?

PERFORMANCE SUDDENLY DEGRADED

- ✓ Check for a recent deploy or config change — correlate with timestamp
- ✓ Profile before assuming the cause — identify the actual bottleneck
- ✓ Check database: any slow query logs, missing indexes, or lock contention?
- ✓ Check external dependencies: is a third-party API responding slowly?
- ✓ Check resource utilization: CPU, memory, connection pool exhaustion?

TEST FAILS INTERMITTENTLY

- ✓ Run the test 50 times in isolation — does it flake alone or only in a suite?
- ✓ Check for time dependencies: any sleep() calls or timeout assumptions?
- ✓ Check for shared state: global variables, database state, file system shared between tests?
- ✓ Check for external dependencies: network calls, real databases in a test that should be mocked?

- ✓ Check for port conflicts if starting a server in the test

WORKS LOCALLY, FAILS IN CI/PRODUCTION

- ✓ Compare environment variables: what's set locally but not in CI?
 - ✓ Compare dependency versions: is the lockfile being used in CI?
 - ✓ Compare OS/platform: macOS dev vs. Linux CI can have path, file permission, or line-ending differences
 - ✓ Check for absolute paths in code — they never transport between environments
 - ✓ Try reproducing in Docker with the CI base image
-

Index & Resources

The complete reference library for going deeper.

No single guide can cover every debugging scenario you will encounter. The books, tools, communities, and specifications listed here are the resources that working engineers return to most often — curated for depth and practical value.

Essential Books

● BEGINNER

- **BEGINNER** *The Pragmatic Programmer* — Andrew Hunt & David Thomas. The origin of rubber duck debugging; timeless debugging and craft principles.
- **BEGINNER** *Debugging: The 9 Indispensable Rules* — David Agans. The most practical, systematic guide to the debugging process ever written.
- **SYSTEMS** *Systems Performance* — Brendan Gregg. The definitive reference for performance analysis, profiling, and flame graphs.
- **DISTRIBUTED** *Designing Distributed Systems* — Brendan Burns. Debugging patterns for distributed architectures.
- **SRE** *Site Reliability Engineering* — Google. The Four Golden Signals; blameless postmortems; monitoring philosophy.
- **SECURITY** *The Art of Software Security Assessment* — Dowd, McDonald, Schuh. Deep source code auditing for security bugs.

— **EBPF** *Learning eBPF* — Liz Rice (O'Reilly, 2023). Hands-on guide to eBPF for observability, net-working, and security tooling.

Tools Index by Category

● BEGINNER

— **ERROR TRACK** **Sentry** (<https://sentry.io>) · **Rollbar** (<https://rollbar.com>) · **Bugsnag** (<https://bugsnag.com>) · **Airbrake** (<https://airbrake.io>)

— **APM** **Datadog** (<https://datadoghq.com>) · **New Relic** (<https://newrelic.com>) · **Dynatrace** (<https://dynatrace.com>) · **Grafana** (<https://grafana.com>)

— **TRACING** **Jaeger** (<https://www.jaegertracing.io>) · **Grafana Tempo** (<https://grafana.com/oss/tempo>) · **Honeycomb** (<https://honeycomb.io>) · **OpenTelemetry** (<https://opentelemetry.io>)

— **AI DEBUG** **LangSmith** (<https://smith.langchain.com>) · **Langfuse** (<https://langfuse.com>) · **Arize Phoenix** (<https://phoenix.arize.com>) · **Braintrust** (<https://braintrust.dev>) · **Helicone** (<https://helicone.ai>)

— **AI AGENTS** **Claude Code** (<https://claude.ai/code>) · **Cursor** (<https://cursor.sh>) · **Aider** (<https://aider.chat>) · **TestSprite** (<https://testsprite.com>) · **Qodo** (<https://qodo.ai>)

— **STATIC ANALYSIS** **ruff** (<https://github.com/astral-sh/ruff>) · **Semgrep** (<https://semgrep.dev>) · **Snyk Code** (<https://snyk.io>) · **CodeQL** (<https://codeql.github.com>) · **SonarQube** (<https://www.sonarqube.org>)

— **NETWORK** **mitmproxy** (<https://mitmproxy.org>) · **Wireshark** (<https://www.wireshark.org>) · **Bruno** (<https://www.usebruno.com>) · **httpie** (<https://httpie.io>)

— **MEMORY** **Valgrind** (<https://valgrind.org>) · **memray** (<https://github.com/bloomberg/memray>) · **AddressSanitizer** (compiler flag) · **Heaptrack** (<https://github.com/KDE/heaptrack>)

— **PROFILING** **Scalene (Python)** (<https://github.com/plasma-umass/scalene>) · **py-spy** (<https://github.com/benfred/py-spy>) · **async-profiler (JVM)** (<https://github.com/asynckti/asynckti>) · **pprof (Go)** (<https://pkg.go.dev/net/http/pprof>) · **Parca** (<https://www.parca.dev>) · **Pyroscope** (<https://pyroscope.io>) · **bpftop** (<https://github.com/Netflix/bpftop>)

— **EBPF** **Beyla** (<https://github.com/grafana/beyla>) · **Odigos** (<https://odigos.io>) · **bpfttrace** (<https://github.com/bpfttrace/bpfttrace>) · **Inspektor Gadget** (<https://www.inspektor-gadget.io>) · **DeepFlow** (<https://deepflow.io>) · **Coroot** (<https://coroot.com>) · **Kyanos** (<https://github.com/hengyoush/kyanos>) · **pwru** (<https://github.com/cilium/pwru>)

— **MCP** **MCP Registry** (<https://modelcontextprotocol.io>) · GitHub MCP · Sentry MCP · Datadog MCP · Supabase MCP · Linear MCP · **SonarQube MCP** (<https://docs.sonarsource.com/sonarqube-mcp-server/>) · **Playwright MCP** (<https://github.com/microsoft/playwright-mcp>) · **Context7 MCP** (<https://context7.com>)



Final Principle

Debugging is not a punishment. It is the work of understanding — the practice of closing the gap between what you believe about a system and what is actually true. Every bug is a lesson about an assumption you didn't know you were making. The engineers who become exceptional debuggers are those who approach that discovery with curiosity rather than frustration.

CITATION

How to Cite This Guide

If you reference this guide in academic work, documentation, or project READMEs, please use one of the formats below. Both are consistent with the output generated by GitHub's "Cite this repository" button and the `cffconvert` (<https://github.com/citation-file-format/cffconvert>) tool from the repository's `CITATION.cff` file.

APA (7th Edition)

```
Made in Jurgistan. (2026). Debugging Field Manual (Version 2026.2.0) [Computer software]. https://Made-in-Jurgistan.github.io/debugging-field-manual/
```

BibTeX

```
@software{Made_in_Jurgistan_Debugging_Field_2026,  
  author = {Made in Jurgistan},  
  month = {7},  
  title = {{Debugging Field Manual}},  
  url = {https://Made-in-Jurgistan.github.io/debugging-field-manual/},  
  version = {2026.2.0},  
  year = {2026}  
}
```



GitHub "Cite this repository" button

A `CITATION.cff` file at the repository root also enables GitHub's built-in "Cite this repository" button (right sidebar on the repo page), which auto-generates APA and BibTeX citations from the same metadata. This button appears on github.com only — it does not render on the published GitHub Pages site, which is why the citation blocks above are included here.

Debugging Field Manual

Complete Edition — 29 Sections · Language-Agnostic · AI-Augmented Workflows

Version 2026.2.0 · Published 2026-07-24 · Updated 2026-07-24 (ISO 8601)

Made in Jurgistan — Complete Edition

Licensed under CC BY-SA 4.0 — Free to share and adapt with attribution

Accessibility: Conforms to WCAG 2.2 AA. Includes keyboard navigation, skip links, focus-visible outlines, reduced-motion support, forced-colors support, and semantic HTML.

Report issues to: github.com/Made-in-Jurgistan/debugging-field-manual/issues

↑ Return to top